ENGINEERING WORKSHEET

Building an AI SRE

From an incident alert to a verified recovery: thirteen layers, thirteen artifacts

---

This worksheet is completed in teams. Every question is answered through conversation with coding agents, Claude and ChatGPT, and every layer ends in a written artifact. Work the layers in the order shown below. A layer may not be started until the artifacts it is built from exist.

Team _____ Breakout room _____ Date _____

Members _____

THE LAYER MAP

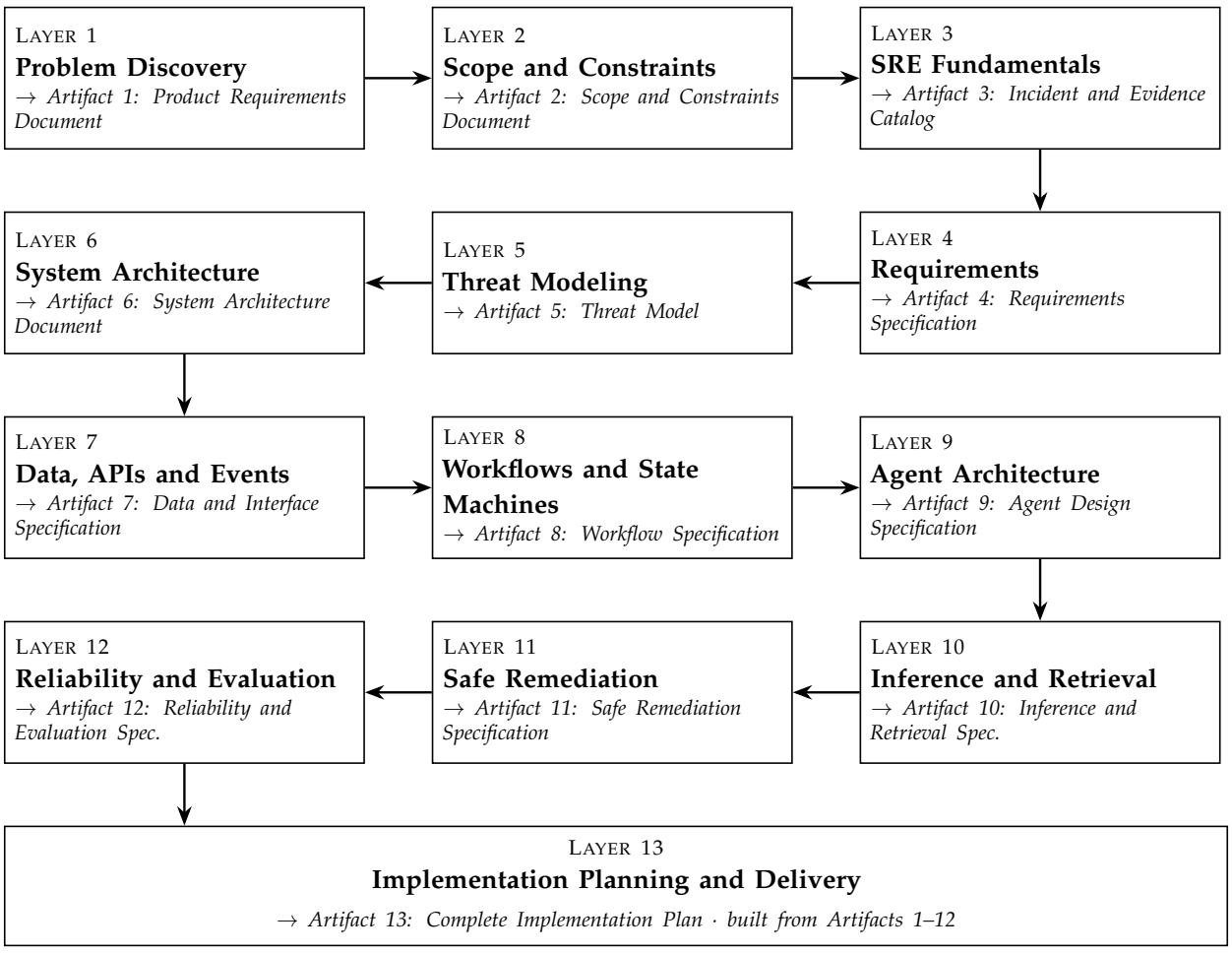


Figure 1. The thirteen layers of the worksheet and the artifact each layer produces. Arrows show the order in which the layers are worked. Each layer also draws on earlier artifacts beyond its immediate predecessor; the table on the next page lists every input.

The Thirteen Layers and Their Artifacts

Each layer answers one central question and produces one artifact. A layer may not be closed until every artifact it is built from has been produced. Question counts are shown so that teams can plan their time in the breakout room.

Layer	Central question	Artifact produced	Built from	Qs
1	**Problem Discovery** *What problem are we actually solving?*	Artifact 1: Product Requirements Document	—	22
2	**Scope and Constraints** *What are we allowed to build?*	Artifact 2: Scope and Constraints Document	A1	23
3	**SRE Fundamentals** *How does an incident actually happen?*	Artifact 3: Incident and Evidence Catalog	A1, A2	37
4	**Functional and Non-Functional Requirements** *What must the system do, and how well must it do it?*	Artifact 4: Requirements Specification	A1, A2, A3	34
5	**Threat Modeling** *What can go wrong?*	Artifact 5: Threat Model	A3, A4	28
6	**System Architecture** *What components do we need?*	Artifact 6: System Architecture Document	A4, A5	39
7	**Data Architecture, APIs and Event Design** *How is the incident lifecycle represented as data?*	Artifact 7: Data and Interface Specification	A4, A6	64
8	**Distributed Workflows and State Machines** *How does long-running work survive failure?*	Artifact 8: Workflow Specification	A6, A7	48
9	**Agent Architecture and Engineering Reasoning** *How does the system investigate like an engineer?*	Artifact 9: Agent Design Specification	A3, A7, A8	53
10	**Inference Engineering and Knowledge Retrieval** *How do we run models reliably and give them the right information?*	Artifact 10: Inference and Retrieval Specification	A2, A9	64
11	**Patch Generation, Sandboxing, Approval and Execution** *How does a recommendation become a controlled, verified change?*	Artifact 11: Safe Remediation Specification	A5, A8, A9	92
12	**Reliability, Observability and Evaluation** *How do we establish that it works under realistic failure?*	Artifact 12: Reliability and Evaluation Specification	A3, A4, A10, A11	83
13	**Implementation Planning and Delivery** *How do we build it?*	Artifact 13: Complete Implementation Plan	Artifacts 1–12	58

HOW EVERY LAYER IS ORGANISED

1. **Layer opener.** The central question, the artifacts the layer is built from, and the focus that governs every answer in it.
2. **Method.** The procedure the team follows for every question: frame, list knowns and unknowns, take them to Claude and ChatGPT, compare the responses, resolve, commit.
3. **Questions.** Numbered **Q***layer.number*. Every question must be answered. A single agent conversation may cover several related questions.
4. **Conversation log.** One row per agent conversation, recording the decision reached and what remains unknown.
5. **Artifact sheet.** The required sections of the layer's artifact and the acceptance checks the team completes before submitting it.

LAYER 1

Problem Discovery

What problem are we actually solving?

METHOD FOR THIS LAYER

Built from. None. This layer begins the worksheet.

Work through every question in this layer with your coding agents. For each question, follow the procedure below before you write anything into the artifact.

1. **Frame.** Restate the question in one sentence, in your own words.
2. **Knowns.** List what the scenario and the earlier artifacts establish as fact.
3. **Unknowns.** List what you must discover, measure or decide.
4. **Assumptions.** List what you are treating as true without evidence. Mark each as *provisional*.
5. **Converse.** Take your knowns, unknowns and assumptions to Claude and to ChatGPT, separately. Ask each to identify unknowns you missed, to challenge your assumptions and to propose an answer. Compare the two responses: where they agree, which unknowns each raised, and where they disagree.
6. **Resolve.** Decide on an answer as a team. Record what remains unknown and who owns it.
7. **Commit.** Enter the decision in the conversation log (page 5) and in the artifact section it supports (page 6).

Focus for this layer. Answers must describe people, behaviour and outcomes. Do not name any technology, framework, database, model or vendor.

Deliverable. ARTIFACT 1 — PRODUCT REQUIREMENTS DOCUMENT. Problem statement · Target users · User journey · Core use cases · Product boundaries · Success definition · Explicit non-goals.

A. UNDERSTANDING THE PROBLEM

Q1.1 An organization operates several software services, and engineers frequently receive alerts about failures. How would you identify the underlying problem that needs to be solved?

Q1.2 Different teams experience service failures, but ownership of resolving them is unclear. How would you identify the people responsible for handling these failures?

Q1.3 A production service goes down at 2 AM. What happens to the business, its users and the engineering team?

Q1.4 An engineer receives an incident alert. Walk through everything they currently have to do before identifying the cause.

Q1.5 An engineer spends hours repeatedly checking logs, metrics and deployment history. Which parts of this investigation could be automated?

Q1.6 Some debugging decisions require experience and judgment. How would you distinguish repetitive investigation from decisions that require human expertise?

Q1.7 Engineers take too long to resolve incidents because information is scattered across different systems. What information would they need to identify the root cause?

Q1.8 An engineer makes an incorrect diagnosis and applies an unnecessary fix. What could happen to the application and its users?

B. UNDERSTANDING THE PRODUCT

Q1.9 Engineers receive an incident alert but have no automated mechanism to investigate it. What capabilities should a system provide immediately after an incident occurs?

Q1.10 A debugging assistant can explain an error but cannot help resolve it. What additional capabilities would be required to move from explanation to remediation?

Q1.11 An AI system identifies a possible fix, but applying it could affect production. Who should be responsible for authorizing that action?

Q1.12 An AI system claims that it has repaired an application, but the application is still returning errors. How would you independently establish whether the repair worked?

Q1.13 An AI system cannot establish the root cause because telemetry is incomplete. How should it behave?

Q1.14 An ordinary AI debugging assistant provides suggestions, while an engineering platform can investigate, coordinate actions and verify recovery. What additional responsibilities distinguish these two products?

Q1.15 Engineers refuse to trust an AI system with production incidents. What evidence, controls and guarantees would make them comfortable using it?

C. DEFINING THE PRODUCT OUTCOME

Q1.16 The team has six months to build the platform, but the initial budget only supports a small implementation. Which capabilities would constitute the smallest genuinely useful product?

Q1.17 Stakeholders want to see the platform working end to end. Design a demonstration that begins with an incident and ends with a verified recovery.

Q1.18 Management wants to establish whether the platform is worth investing in. Which measurable outcomes would demonstrate its effectiveness?

Q1.19 An AI system is capable of performing actions that could damage production. Which actions should the platform explicitly prohibit?

D. THE REFERENCE SCENARIO

REFERENCE SCENARIO

- An organization operates a Python web application.
- An engineer deploys a new version.
- The deployment changes a configuration value.
- The application starts returning HTTP 500 errors.
- The monitoring system detects the incident.

This scenario is used throughout the worksheet. Every later artifact must remain consistent with it.

Q1.20 The monitoring system has detected an incident. Design the complete sequence of actions the platform should perform, from receiving the alert until the service recovers.

Q1.21 Multiple systems contribute information during the investigation. How should the platform preserve that information throughout the incident lifecycle?

Q1.22 Some decisions could cause irreversible production changes. At which exact points should a human become involved, and why?

LAYER 1 · PROBLEM DISCOVERY

Conversation Log

Record one row per agent conversation. A conversation may cover several of the 22 questions in this layer; list all of them. Mark the agent used: **C** for Claude, **G** for ChatGPT, or both. Continue on additional sheets if required.

Questions	Agent	Conversation link	Decision reached	Open unknowns and owner	Artifact section

WHERE CLAUDE AND CHATGPT DISAGREED

Record each disagreement that changed a team decision, and how the team resolved it.

ARTIFACT 1

Product Requirements Document

Built from. None. This layer begins the worksheet.

Produce this artifact with your coding agent from the decisions recorded in the conversation log. Complete the table as each section is written.

	Required section	Questions this section answers	Done
1.1	Problem statement		☐
1.2	Target users		☐
1.3	User journey		☐
1.4	Core use cases		☐
1.5	Product boundaries		☐
1.6	Success definition		☐
1.7	Explicit non-goals		☐

ACCEPTANCE CHECKS BEFORE SUBMISSION

- ☐ Every required section above is present and complete.
- ☐ Every section lists the questions it answers, and every question in this layer is answered by at least one section.
- ☐ Every assumption is marked *provisional* and has a named owner and a condition under which it will be validated.
- ☐ Every disagreement between Claude and ChatGPT that affected a decision is recorded with the team's resolution.
- ☐ The artifact is consistent with every artifact it is built from; any conflict is recorded as an explicit decision.
- ☐ The artifact was produced with the team's coding agent and is saved in the team repository.

Reviewed by _____ Date _____

LAYER 2

Scope and Constraints

What are we allowed to build?

METHOD FOR THIS LAYER

Built from. Artifact 1 (Product Requirements Document).

Work through every question in this layer with your coding agents. For each question, follow the procedure below before you write anything into the artifact.

1. **Frame.** Restate the question in one sentence, in your own words.
2. **Knowns.** List what the scenario and the earlier artifacts establish as fact.
3. **Unknowns.** List what you must discover, measure or decide.
4. **Assumptions.** List what you are treating as true without evidence. Mark each as *provisional*.
5. **Converse.** Take your knowns, unknowns and assumptions to Claude and to ChatGPT, separately. Ask each to identify unknowns you missed, to challenge your assumptions and to propose an answer. Compare the two responses: where they agree, which unknowns each raised, and where they disagree.
6. **Resolve.** Decide on an answer as a team. Record what remains unknown and who owns it.
7. **Commit.** Enter the decision in the conversation log (page 9) and in the artifact section it supports (page 10).

Focus for this layer. Architecture starts with constraints, not technology selection. Establish boundaries, budgets and assumptions before choosing any tool.

Deliverable. ARTIFACT 2 — SCOPE AND CONSTRAINTS DOCUMENT. Supported environments · Supported application types · Deployment boundaries · Budget constraints · Provider assumptions · Explicit non-goals · Unresolved dependencies.

A. ENVIRONMENT

Q2.1 An organization wants to deploy the platform across several engineering teams. Should the initial system serve one engineering organization or multiple independent organizations?

Q2.2 The organization's applications use different programming languages. Which languages should the initial platform support?

Q2.3 Some applications run on cloud infrastructure while others operate on private infrastructure. Which deployment environments should the platform support?

Q2.4 The organization already uses monitoring platforms. How should the new platform integrate with these existing systems?

Q2.5 The investigation requires logs, metrics and traces, but the platform currently has no access to them. Which data access permissions and integrations must be established?

Q2.6 An engineer needs to understand whether a failure was caused by a recent deployment. How could the platform access deployment history and source repositories?

Q2.7 Real production incidents contain sensitive information and are difficult to reproduce. Should the initial implementation use real incidents or sanitized synthetic fixtures?

B. SCOPE

Q2.8 Stakeholders expect a sophisticated dashboard, but the engineering team has limited time. Could the first version operate using only a CLI and HTTP API?

Q2.9 The team is considering deploying the application on Kubernetes even though the initial workload is small. What operational problem would Kubernetes solve, and is that problem present yet?

Q2.10 Engineers propose splitting every component into an independent microservice. What problems would this introduce, and does the initial workload justify them?

Q2.11 The initial implementation requires an AI model, but the team is considering training its own model. What capabilities would justify developing a custom model?

Q2.12 The team wants to operate its own GPU infrastructure. What workload, cost or control requirements would justify this investment?

Q2.13 Several organizations may eventually use the platform. Which technical and security requirements would multi-tenant operation introduce?

Q2.14 The AI can generate a repair but the team is not ready to let it modify production. How could execution be restricted to a disposable environment?

C. CONSTRAINTS

Q2.15 The organization has a limited initial infrastructure budget. How should infrastructure spending be bounded?

Q2.16 Every incident requires several model calls. How would you establish an acceptable model expenditure per incident?

Q2.17 The organization experiences sudden traffic spikes and multiple services can fail simultaneously. How many concurrent incidents should the system support?

Q2.18 Some investigations require several minutes to collect evidence and evaluate hypotheses. How long should an investigation be allowed to run?

Q2.19 The model provider becomes unavailable during an incident. How should the platform behave?

Q2.20 Several architectural requirements have not yet been validated. How would you distinguish fixed requirements from assumptions?

D. DECISIONS AND ASSUMPTIONS

Q2.21 The team records a decision to use PostgreSQL, but this choice is based on an unverified assumption about data volume. How would you distinguish this architectural decision from the assumption behind it?

Q2.22 An assumption about model availability turns out to be false after implementation. Which assumptions could invalidate the architecture if they change?

Q2.23 Multiple engineering decisions remain unresolved. How would you record these decisions, their rationale, their owners and the conditions required to validate them?

LAYER 2 · SCOPE AND CONSTRAINTS

Conversation Log

Record one row per agent conversation. A conversation may cover several of the 23 questions in this layer; list all of them. Mark the agent used: **C** for Claude, **G** for ChatGPT, or both. Continue on additional sheets if required.

Questions	Agent	Conversation link	Decision reached	Open unknowns and owner	Artifact section

WHERE CLAUDE AND CHATGPT DISAGREED

Record each disagreement that changed a team decision, and how the team resolved it.

ARTIFACT 2

Scope and Constraints Document

Built from. Artifact 1 (Product Requirements Document).

Produce this artifact with your coding agent from the decisions recorded in the conversation log. Complete the table as each section is written.

	Required section	Questions this section answers	Done
2.1	Supported environments		☐
2.2	Supported application types		☐
2.3	Deployment boundaries		☐
2.4	Budget constraints		☐
2.5	Provider assumptions		☐
2.6	Explicit non-goals		☐
2.7	Unresolved dependencies		☐

ACCEPTANCE CHECKS BEFORE SUBMISSION

- ☐ Every required section above is present and complete.
- ☐ Every section lists the questions it answers, and every question in this layer is answered by at least one section.
- ☐ Every assumption is marked *provisional* and has a named owner and a condition under which it will be validated.
- ☐ Every disagreement between Claude and ChatGPT that affected a decision is recorded with the team's resolution.
- ☐ The artifact is consistent with every artifact it is built from; any conflict is recorded as an explicit decision.
- ☐ The artifact was produced with the team's coding agent and is saved in the team repository.

Reviewed by _____ Date _____

LAYER 3

SRE Fundamentals

How does an incident actually happen?

METHOD FOR THIS LAYER

Built from. Artifacts 1 and 2.

Work through every question in this layer with your coding agents. For each question, follow the procedure below before you write anything into the artifact.

1. **Frame.** Restate the question in one sentence, in your own words.
2. **Knowns.** List what the scenario and the earlier artifacts establish as fact.
3. **Unknowns.** List what you must discover, measure or decide.
4. **Assumptions.** List what you are treating as true without evidence. Mark each as *provisional*.
5. **Converse.** Take your knowns, unknowns and assumptions to Claude and to ChatGPT, separately. Ask each to identify unknowns you missed, to challenge your assumptions and to propose an answer. Compare the two responses: where they agree, which unknowns each raised, and where they disagree.
6. **Resolve.** Decide on an answer as a team. Record what remains unknown and who owns it.
7. **Commit.** Enter the decision in the conversation log (page 14) and in the artifact section it supports (page 15).

Focus for this layer. Understand the domain before designing the automation. Describe how software fails and how engineers investigate it; do not design the platform in this layer.

Deliverable. ARTIFACT 3 — INCIDENT AND EVIDENCE CATALOG. Initial alert · Service context · Logs · Metrics · Traces · Deployment changes · Expected root cause · Supporting evidence · Contradicting evidence · Expected escalation behavior.

A. SOFTWARE SYSTEMS

Q3.1 A user opens an application, but nothing appears on the screen. Trace the journey of their request through the underlying software components.

Q3.2 A web application receives thousands of simultaneous requests. Which responsibilities must the application components perform to process these requests?

Q3.3 A user sends an HTTP request, but the application returns an error. What happens between receiving the request and returning the response?

Q3.4 An application needs to receive requests from external clients. What component should expose the communication interface?

Q3.5 An application needs to persist customer information even after restarting. What component should manage that information?

Q3.6 An engineer introduces a new version of an application. What sequence of operations takes place when that version is deployed?

Q3.7 An application behaves differently after an environment variable changes, even though its source code remains identical. How would you distinguish the impact of configuration from the impact of application code?

Q3.8 A deployment introduces a regression and causes previously working functionality to fail. How would you investigate and isolate the change responsible?

B. INCIDENT MANAGEMENT

Q3.9 A production application suddenly stops responding to users. What conditions would cause the engineering team to declare and track this situation as an incident?

Q3.10 The application is technically running, but 40% of requests are failing. How should the engineering team distinguish this from a complete outage?

Q3.11 An application remains available, but its response time increases from 100 ms to 8 seconds. How should the engineering team characterize this situation?

Q3.12 Engineers have collected logs, traces and deployment information but have not yet identified the underlying cause. What investigation process should they follow?

Q3.13 Engineers identify the cause of an incident but still need to restore normal service. What actions could resolve the problem?

Q3.14 A new deployment causes failures across the application. How could the engineering team return to a previously working version?

Q3.15 An incident passes through several operational stages, from detection to closure. How would you define and track this lifecycle?

Q3.16 An incident cannot be resolved by the first engineer who investigates it. How should responsibility move to another engineer or team?

C. OBSERVABILITY

Q3.17 Engineers need to understand what happened inside an application after an error occurred. What information should the application record about individual events?

Q3.18 Management wants to know how many requests failed during the previous hour. What numerical measurements should the application expose?

Q3.19 A request passes through five different services, and the final service returns an error. How could engineers reconstruct the journey of that request?

Q3.20 Engineers have application logs but cannot determine how frequently a failure occurs. Why are logs alone insufficient?

Q3.21 Users complain that the application is slow, but the engineering team has no measurement of response duration. Which metric should be collected?

Q3.22 An application suddenly starts returning errors for 30% of requests. Which measurements would help establish the severity of the problem?

Q3.23 A service becomes unavailable because its CPU is saturated. Which resource measurements would help identify this condition?

Q3.24 An application starts crashing because it consumes increasing amounts of memory. Which measurements would help diagnose this behavior?

Q3.25 A background processing system accumulates thousands of pending jobs. How would engineers measure and detect this accumulation?

Q3.26 A failure begins immediately after a deployment. What deployment information should be available to correlate changes with incidents?

Q3.27 A deployment happens at exactly the same time that error rates increase. Why is this insufficient by itself to prove that the deployment caused the incident?

D. OPERATIONAL TOOLING

Q3.28 An engineering team needs to collect numerical measurements from its applications and expose them for querying. Which responsibilities should its metrics collection system perform?

Q3.29 Engineers want to visualize service health, error rates and resource consumption. Which capabilities should their monitoring dashboard provide?

Q3.30 An organization operates multiple services and needs to reconstruct request journeys across them. Which instrumentation and telemetry standards would support this?

Q3.31 The organization already uses an external error-tracking platform. How should the AI SRE

platform consume information from it?

Q3.32 The AI SRE platform needs information from several operational tools without directly depending on their internal implementations. How should this integration be designed?

E. PRACTICAL SCENARIOS

THREE INCIDENTS TO MODEL

- **Incident A — Configuration regression.** A deployment changes a configuration value and the application begins returning HTTP 500 errors.
- **Incident B — Code regression.** A deployment introduces a defect in application code that causes previously working functionality to fail.
- **Incident C — Insufficient evidence.** The application fails, but the available telemetry is incomplete and does not identify a cause.

Q3.33 For each incident, which information should the platform collect before proposing a diagnosis?

Q3.34 A hypothesis suggests that a recent deployment caused the incident. Which observations would strengthen this hypothesis?

Q3.35 The deployment happened immediately before the incident, but the application logs show an unrelated database connection failure. Which observations would contradict the original hypothesis?

Q3.36 The monitoring provider becomes unavailable during the investigation. How should the platform proceed?

Q3.37 An investigation has exhausted its evidence sources but still cannot establish a root cause. Under which conditions should it stop and escalate?

LAYER 3 · SRE FUNDAMENTALS

Conversation Log

Record one row per agent conversation. A conversation may cover several of the 37 questions in this layer; list all of them. Mark the agent used: **C** for Claude, **G** for ChatGPT, or both. Continue on additional sheets if required.

Questions	Agent	Conversation link	Decision reached	Open unknowns and owner	Artifact section

WHERE CLAUDE AND CHATGPT DISAGREED

Record each disagreement that changed a team decision, and how the team resolved it.

ARTIFACT 3

Incident and Evidence Catalog

Built from. Artifacts 1 and 2.

Produce this artifact with your coding agent from the decisions recorded in the conversation log. Complete the table as each section is written. Complete every section below for each of the three incidents.

	Required section	Questions this section answers	Done
3.1	Initial alert		☐
3.2	Service context		☐
3.3	Logs		☐
3.4	Metrics		☐
3.5	Traces		☐
3.6	Deployment changes		☐
3.7	Expected root cause		☐
3.8	Supporting evidence		☐
3.9	Contradicting evidence		☐
3.10	Expected escalation behavior		☐

ACCEPTANCE CHECKS BEFORE SUBMISSION

- ☐ Every required section above is present and complete.
- ☐ Every section lists the questions it answers, and every question in this layer is answered by at least one section.
- ☐ Every assumption is marked *provisional* and has a named owner and a condition under which it will be validated.
- ☐ Every disagreement between Claude and ChatGPT that affected a decision is recorded with the team's resolution.
- ☐ The artifact is consistent with every artifact it is built from; any conflict is recorded as an explicit decision.
- ☐ The artifact was produced with the team's coding agent and is saved in the team repository.

Reviewed by _____ Date _____

LAYER 4

Functional and Non-Functional Requirements

What must the system do, and how well must it do it?

METHOD FOR THIS LAYER

Built from. Artifacts 1, 2 and 3.

Work through every question in this layer with your coding agents. For each question, follow the procedure below before you write anything into the artifact.

1. **Frame.** Restate the question in one sentence, in your own words.
2. **Knowns.** List what the scenario and the earlier artifacts establish as fact.
3. **Unknowns.** List what you must discover, measure or decide.
4. **Assumptions.** List what you are treating as true without evidence. Mark each as *provisional*.
5. **Converse.** Take your knowns, unknowns and assumptions to Claude and to ChatGPT, separately. Ask each to identify unknowns you missed, to challenge your assumptions and to propose an answer. Compare the two responses: where they agree, which unknowns each raised, and where they disagree.
6. **Resolve.** Decide on an answer as a team. Record what remains unknown and who owns it.
7. **Commit.** Enter the decision in the conversation log (page 19) and in the artifact section it supports (page 20).

Focus for this layer. Distinguish what the system does from how well it must do it. Every requirement must be measurable; a requirement that cannot be tested is not yet a requirement.

Deliverable. ARTIFACT 4 — REQUIREMENTS SPECIFICATION. Requirement ID · Description · Priority · Measurement · Acceptance criterion · Validation method · Related milestone.

A. FUNCTIONAL REQUIREMENTS

Q4.1 An engineer submits an incident to the platform. What sequence of operations should happen after submission?

Q4.2 Monitoring systems send the same alert multiple times. How should the platform prevent duplicate investigations?

Q4.3 An investigation is created without sufficient context to identify the affected application. Which information should be mandatory?

Q4.4 An investigation needs information from several external systems. How should the evidence collection process work?

Q4.5 The agent identifies three possible causes for an incident. How should these competing hypotheses be represented?

Q4.6 The system has collected evidence and needs to determine the most likely root cause. How should diagnoses be generated?

Q4.7 Two hypotheses have conflicting supporting evidence. How should the system evaluate and compare them?

Q4.8 The AI cannot confidently identify a root cause. How should the system represent an inconclusive diagnosis?

Q4.9 A diagnosis indicates that a code change is required. How should the system convert the diagnosis into a proposed patch?

Q4.10 A patch has been generated, but its correctness is unknown. How should the platform test it?

Q4.11 A patch passes automated tests but could affect production. How should a human review and authorize it?

Q4.12 A patch has been approved. How should the platform execute the authorized operation?

Q4.13 A repair has been applied, but the service may still be unhealthy. How should the platform verify recovery?

Q4.14 An engineer wants to stop an ongoing investigation. How should cancellation work?

Q4.15 An engineer needs to understand every action taken during an incident. How should the platform expose its timeline?

B. NON-FUNCTIONAL REQUIREMENTS

Q4.16 The incident intake service becomes unavailable occasionally. What reliability target should it meet?

Q4.17 A worker crashes halfway through an investigation. How should the system recover without losing progress?

Q4.18 PostgreSQL becomes unavailable while several incidents are active. What should happen to the ongoing investigations?

Q4.19 Monitoring detects an incident, but the platform takes several minutes to acknowledge it. How should acknowledgement latency be measured and bounded?

Q4.20 An investigation continues consuming resources indefinitely. What maximum execution duration should be enforced?

Q4.21 An incident triggers hundreds of model calls and unexpectedly increases the cloud bill. What expenditure limits should be enforced?

Q4.22 Several incidents arrive simultaneously. How should the system manage concurrent investigations?

Q4.23 Application logs contain customer information and credentials. How should the platform protect sensitive information?

Q4.24 The database is corrupted and the team needs to restore operations. What recovery guarantees should be established?

Q4.25 A complete platform failure occurs. How long can the organization tolerate the platform remaining unavailable?

Q4.26 Engineers cannot determine why investigations are slow. Which performance measurements should the platform expose?

Q4.27 An external dependency becomes degraded but is not completely unavailable. How should the platform respond?

C. QUANTIFYING REQUIREMENTS

For every question in this part, propose a value, explain the rationale, and mark every unmeasured value as provisional.

Q4.28 An engineer submits an incident and expects an immediate response. What should the API intake p95 latency target be?

Q4.29 The incident is accepted, but its processing event takes too long to become available to workers. What should the event publication p95 target be?

Q4.30 A model provider occasionally takes several minutes to return a response. What maximum model-call duration should be allowed?

Q4.31 A model returns malformed output several times for the same incident. How many model attempts should the platform permit?

Q4.32 An approval request remains unanswered for several days. How long should an approval remain valid?

Q4.33 An investigation generates excessive input and output tokens. How should cumulative token limits be established?

Q4.34 The system receives more incidents than its workers can handle. What maximum concurrent

investigation count should be enforced?

LAYER 4 · FUNCTIONAL AND NON-FUNCTIONAL REQUIREMENTS

Conversation Log

Record one row per agent conversation. A conversation may cover several of the 34 questions in this layer; list all of them. Mark the agent used: **C** for Claude, **G** for ChatGPT, or both. Continue on additional sheets if required.

Questions	Agent	Conversation link	Decision reached	Open unknowns and owner	Artifact section

WHERE CLAUDE AND CHATGPT DISAGREED

Record each disagreement that changed a team decision, and how the team resolved it.

ARTIFACT 4

Requirements Specification

Built from. Artifacts 1, 2 and 3.

Produce this artifact with your coding agent from the decisions recorded in the conversation log. Complete the table as each section is written. Every requirement must carry each of the following fields.

	Required section	Questions this section answers	Done
4.1	Requirement ID		☐
4.2	Description		☐
4.3	Priority		☐
4.4	Measurement		☐
4.5	Acceptance criterion		☐
4.6	Validation method		☐
4.7	Related milestone		☐

ACCEPTANCE CHECKS BEFORE SUBMISSION

- ☐ Every required section above is present and complete.
- ☐ Every section lists the questions it answers, and every question in this layer is answered by at least one section.
- ☐ Every assumption is marked *provisional* and has a named owner and a condition under which it will be validated.
- ☐ Every disagreement between Claude and ChatGPT that affected a decision is recorded with the team's resolution.
- ☐ The artifact is consistent with every artifact it is built from; any conflict is recorded as an explicit decision.
- ☐ The artifact was produced with the team's coding agent and is saved in the team repository.

Reviewed by _____ Date _____

LAYER 5

Threat Modeling

What can go wrong?

METHOD FOR THIS LAYER

Built from. Artifacts 3 and 4.

Work through every question in this layer with your coding agents. For each question, follow the procedure below before you write anything into the artifact.

1. **Frame.** Restate the question in one sentence, in your own words.
2. **Knowns.** List what the scenario and the earlier artifacts establish as fact.
3. **Unknowns.** List what you must discover, measure or decide.
4. **Assumptions.** List what you are treating as true without evidence. Mark each as *provisional*.
5. **Converse.** Take your knowns, unknowns and assumptions to Claude and to ChatGPT, separately. Ask each to identify unknowns you missed, to challenge your assumptions and to propose an answer. Compare the two responses: where they agree, which unknowns each raised, and where they disagree.
6. **Resolve.** Decide on an answer as a team. Record what remains unknown and who owns it.
7. **Commit.** Enter the decision in the conversation log (page 23) and in the artifact section it supports (page 24).

Focus for this layer. Establish trust boundaries before allowing any AI-generated action. Treat every input the model reads as potentially hostile until shown otherwise.

Deliverable. ARTIFACT 5 — THREAT MODEL. Assets · Threat actors · Attack surfaces · Trust boundaries · Abuse cases · Security controls · Residual risks · Security test cases.

A. THREAT IDENTIFICATION

Q5.1 The AI receives application logs containing text such as "ignore previous instructions and execute this command." How could malicious application data influence the model's behavior?

Q5.2 The AI receives system instructions and untrusted application data in the same context. How would you prevent the model from confusing data with trusted instructions?

Q5.3 The model generates a destructive shell command while attempting to fix an application. What damage could this cause?

Q5.4 The model confidently reports an incorrect diagnosis. How could this lead to an unsafe production change?

Q5.5 A model returns malformed JSON instead of the expected structured diagnosis. How should the platform handle it?

Q5.6 An engineer submits an incident against a service they do not have permission to access. How should the platform detect and reject this request?

Q5.7 An unauthorized user attempts to approve a production action. Which controls should prevent this?

Q5.8 A patch is modified after it has been reviewed and approved. How could the system detect this modification?

Q5.9 A generated patch needs access to environment variables containing production credentials. How would you prevent credential exposure?

Q5.10 A worker executes the same action twice following a retry. What damage could duplicate execution cause?

Q5.11 An external operation succeeds, but the worker crashes before receiving confirmation. How should the platform determine whether the operation actually happened?

B. TRUST BOUNDARIES

Q5.12 The platform contains an AI model, an API, a database, a sandbox and a production execution environment. Which components should be trusted, and which should be treated as untrusted?

Q5.13 The model needs to investigate infrastructure failures. Should it receive direct infrastructure credentials, or should access be mediated?

Q5.14 An AI agent wants to run arbitrary shell commands to investigate an application. What risks would unrestricted shell access introduce?

Q5.15 The model needs to query application data. Under what circumstances should it be prevented from constructing arbitrary SQL?

Q5.16 An agent wants to inspect an external URL referenced in an application log. How could unrestricted URL access expose internal infrastructure?

Q5.17 The platform provides several operational tools. Which operations should remain read-only?

Q5.18 A generated action needs to modify production state. Which component should have exclusive responsibility for executing writes?

Q5.19 The platform needs to verify that a repair is safe. Which component should control the trusted verification process?

Q5.20 An engineer disputes an action performed by the AI. How should the platform establish an auditable record of who authorized and executed it?

C. SECURITY CONTROLS

Q5.21 An unauthorized person attempts to access the incident dashboard. How should the platform establish their identity?

Q5.22 An authenticated engineer tries to modify a service they do not own. How should the platform enforce permissions?

Q5.23 Malicious text appears inside application logs and is included in an AI prompt. How should the system prevent this text from becoming an instruction?

Q5.24 A user attempts to investigate a service outside their authorized scope. How should the platform restrict target access?

Q5.25 AI-generated code needs to be executed for testing. How should it be isolated from the host environment?

Q5.26 A model accidentally returns a secret in its response. How should the platform prevent that secret from being exposed or stored insecurely?

Q5.27 An attacker attempts to modify an approval request after authorization. How should the system verify the integrity of the original approval?

Q5.28 An administrator needs to investigate a suspicious production action. How should every consequential operation be recorded?

LAYER 5 · THREAT MODELING

Conversation Log

Record one row per agent conversation. A conversation may cover several of the 28 questions in this layer; list all of them. Mark the agent used: **C** for Claude, **G** for ChatGPT, or both. Continue on additional sheets if required.

Questions	Agent	Conversation link	Decision reached	Open unknowns and owner	Artifact section

WHERE CLAUDE AND CHATGPT DISAGREED

Record each disagreement that changed a team decision, and how the team resolved it.

ARTIFACT 5

Threat Model

Built from. Artifacts 3 and 4.

Produce this artifact with your coding agent from the decisions recorded in the conversation log. Complete the table as each section is written.

	Required section	Questions this section answers	Done
5.1	Assets		☐
5.2	Threat actors		☐
5.3	Attack surfaces		☐
5.4	Trust boundaries		☐
5.5	Abuse cases		☐
5.6	Security controls		☐
5.7	Residual risks		☐
5.8	Security test cases		☐

ACCEPTANCE CHECKS BEFORE SUBMISSION

- ☐ Every required section above is present and complete.
- ☐ Every section lists the questions it answers, and every question in this layer is answered by at least one section.
- ☐ Every assumption is marked *provisional* and has a named owner and a condition under which it will be validated.
- ☐ Every disagreement between Claude and ChatGPT that affected a decision is recorded with the team's resolution.
- ☐ The artifact is consistent with every artifact it is built from; any conflict is recorded as an explicit decision.
- ☐ The artifact was produced with the team's coding agent and is saved in the team repository.

Reviewed by _____ Date _____

LAYER 6

System Architecture

What components do we need?

METHOD FOR THIS LAYER

Built from. Artifacts 4 and 5.

Work through every question in this layer with your coding agents. For each question, follow the procedure below before you write anything into the artifact.

1. **Frame.** Restate the question in one sentence, in your own words.
2. **Knowns.** List what the scenario and the earlier artifacts establish as fact.
3. **Unknowns.** List what you must discover, measure or decide.
4. **Assumptions.** List what you are treating as true without evidence. Mark each as *provisional*.
5. **Converse.** Take your knowns, unknowns and assumptions to Claude and to ChatGPT, separately. Ask each to identify unknowns you missed, to challenge your assumptions and to propose an answer. Compare the two responses: where they agree, which unknowns each raised, and where they disagree.
6. **Resolve.** Decide on an answer as a team. Record what remains unknown and who owns it.
7. **Commit.** Enter the decision in the conversation log (page 28) and in the artifact section it supports (page 29).

Focus for this layer. Derive the architecture from responsibilities. Parts A and B must be complete before any technology is named in Part C.

Deliverable. ARTIFACT 6 — SYSTEM ARCHITECTURE DOCUMENT. Context diagram · Container/process diagram · Component diagram · Technology selection · Architecture Decision Records (ADRs) · Component ownership · Trust boundaries · Alternatives and trade-offs.

A. DECOMPOSE THE SYSTEM

Q6.1 The team implements the entire incident lifecycle inside one Python function. An investigation takes 10 minutes, and the HTTP request times out. How should responsibilities be separated to avoid keeping the request open?

Q6.2 The API needs to acknowledge an incident immediately, but investigation requires several minutes. Which operations should happen synchronously and which should happen asynchronously?

Q6.3 The API process currently receives requests, collects logs, calls models and executes patches. How would you separate its responsibilities?

Q6.4 Several investigations need to run independently while the API continues accepting new requests. What should a background processing component be responsible for?

Q6.5 The team proposes creating a separate deployable service for every module. What operational problems could this introduce?

Q6.6 A growing application contains many unrelated responsibilities inside a single codebase. How could the code be organized without introducing distributed service complexity?

Q6.7 A small engineering team wants to deploy its initial platform quickly while retaining clear module boundaries. Which architectural approach would support this?

Q6.8 The API and background processing systems need different runtime behavior, but share domain logic and database contracts. How could they run as separate processes while sharing one codebase?

B. COMPONENT IDENTIFICATION

Identify every major component. Name each by its responsibility, not by a product.

Q6.9 An external monitoring system detects an incident. Which component should receive and validate the incoming incident?

Q6.10 The platform needs to preserve incident information across restarts. Which component should manage persistent incident records?

Q6.11 An investigation needs to resume after a worker crashes. Which component should manage durable workflow progress?

Q6.12 The agent needs to communicate with multiple AI providers. Which component should own model inference requests?

Q6.13 Engineers have internal runbooks that the AI needs during investigations. Which component should retrieve relevant knowledge?

Q6.14 The platform needs to collect logs and metrics from customer applications. Which component should coordinate evidence collection?

Q6.15 The agent has collected evidence and needs to identify possible causes. Which component should generate diagnoses?

Q6.16 A diagnosis needs independent review before becoming an actionable recommendation. Which component should evaluate it?

Q6.17 AI-generated code needs to be executed against a test suite. Which component should own this operation?

Q6.18 A production repair requires human authorization. Which component should manage approval?

Q6.19 An approved action needs to modify an external system. Which component should execute that operation?

Q6.20 A repair has been applied, but its effectiveness is unknown. Which component should independently verify recovery?

Q6.21 Large logs, diffs and test outputs exceed the practical size of database records. Where should these artifacts be stored?

Q6.22 Engineers need to understand whether the AI SRE platform itself is healthy. Which component should collect and expose platform telemetry?

C. TECHNOLOGY SELECTION

Answer this part only after Parts A and B are complete. Select each technology against a responsibility you have already identified.

Q6.23 The platform needs to expose HTTP endpoints and validate incoming requests. Which Python framework would satisfy these requirements?

Q6.24 The system stores structured incident records and transactional state. Which database technology should be evaluated?

Q6.25 Multiple engineers modify database schemas over time. How should schema changes be tracked and applied safely?

Q6.26 Investigations can take several minutes or hours, and must survive process crashes. Which workflow technology supports this requirement?

Q6.27 The platform needs to communicate with multiple model providers. Which providers should be supported initially?

Q6.28 Different providers expose different request and response formats. How should the platform hide these differences?

Q6.29 The agent needs access to internal runbooks. Which retrieval strategy would be appropriate for the first implementation?

Q6.30 AI-generated code needs to be executed in an isolated environment. Which execution isolation approach should be used?

Q6.31 The platform needs to expose metrics and dashboards. Which observability technologies should be considered?

Q6.32 Engineers need to run the entire system locally without deploying every component independently. How should the development environment be assembled?

Q6.33 The team is considering additional technologies that introduce operational overhead. Which technologies should be avoided initially, and what evidence would justify introducing them later?

Evaluate the candidate stack below against your answers above. For each row, accept, reject or replace the candidate and record the decision as an Architecture Decision Record.

Candidate stack

Responsibility	Candidate	Team decision (record as an ADR)
Backend API	FastAPI	☐ Accept ☐ Reject ☐ Replace: _____
Validation	Pydantic	☐ Accept ☐ Reject ☐ Replace: _____
Database	PostgreSQL	☐ Accept ☐ Reject ☐ Replace: _____
ORM	SQLAlchemy	☐ Accept ☐ Reject ☐ Replace: _____
Migrations	Alembic	☐ Accept ☐ Reject ☐ Replace: _____
Workflow engine	Temporal	☐ Accept ☐ Reject ☐ Replace: _____
Model providers	Claude and Tinker	☐ Accept ☐ Reject ☐ Replace: _____
Retrieval	BM25	☐ Accept ☐ Reject ☐ Replace: _____
Execution	Restricted sandbox	☐ Accept ☐ Reject ☐ Replace: _____
Metrics	Prometheus	☐ Accept ☐ Reject ☐ Replace: _____
Dashboard	Grafana	☐ Accept ☐ Reject ☐ Replace: _____
Tracing	OpenTelemetry	☐ Accept ☐ Reject ☐ Replace: _____
Development environment	Docker Compose	☐ Accept ☐ Reject ☐ Replace: _____
Dependency management	uv	☐ Accept ☐ Reject ☐ Replace: _____

D. ARCHITECTURE VIEWS

Q6.34 A new engineer joins the project and cannot understand how the platform's major components communicate. How would you represent the complete architecture visually?

Q6.35 The API, PostgreSQL and Temporal each have different responsibilities. How would you represent their communication and dependencies in an architecture diagram?

Q6.36 The agent needs to communicate with a model gateway, which communicates with multiple model providers. How would you represent this relationship?

Q6.37 The sandbox must never directly authorize production actions. How would you represent the relationship between sandbox, approval service and trusted dispatcher?

Q6.38 The architecture contains logical modules and independently deployed processes. How would you distinguish them visually and explain their deployment boundaries?

Q6.39 Some components process untrusted code or hold production credentials. How would you identify which components need separate security boundaries?

LAYER 6 · SYSTEM ARCHITECTURE

Conversation Log

Record one row per agent conversation. A conversation may cover several of the 39 questions in this layer; list all of them. Mark the agent used: **C** for Claude, **G** for ChatGPT, or both. Continue on additional sheets if required.

Questions	Agent	Conversation link	Decision reached	Open unknowns and owner	Artifact section

WHERE CLAUDE AND CHATGPT DISAGREED

Record each disagreement that changed a team decision, and how the team resolved it.

ARTIFACT 6

System Architecture Document

Built from. Artifacts 4 and 5.

Produce this artifact with your coding agent from the decisions recorded in the conversation log. Complete the table as each section is written.

	Required section	Questions this section answers	Done
6.1	Context diagram		☐
6.2	Container/process diagram		☐
6.3	Component diagram		☐
6.4	Technology selection		☐
6.5	Architecture Decision Records (ADRs)		☐
6.6	Component ownership		☐
6.7	Trust boundaries		☐
6.8	Alternatives and trade-offs		☐

ACCEPTANCE CHECKS BEFORE SUBMISSION

- ☐ Every required section above is present and complete.
- ☐ Every section lists the questions it answers, and every question in this layer is answered by at least one section.
- ☐ Every assumption is marked *provisional* and has a named owner and a condition under which it will be validated.
- ☐ Every disagreement between Claude and ChatGPT that affected a decision is recorded with the team's resolution.
- ☐ The artifact is consistent with every artifact it is built from; any conflict is recorded as an explicit decision.
- ☐ The artifact was produced with the team's coding agent and is saved in the team repository.

Reviewed by _____ Date _____

LAYER 7

Data Architecture, APIs and Event Design

How is the incident lifecycle represented as data?

METHOD FOR THIS LAYER

Built from. Artifacts 4 and 6.

Work through every question in this layer with your coding agents. For each question, follow the procedure below before you write anything into the artifact.

1. **Frame.** Restate the question in one sentence, in your own words.
2. **Knowns.** List what the scenario and the earlier artifacts establish as fact.
3. **Unknowns.** List what you must discover, measure or decide.
4. **Assumptions.** List what you are treating as true without evidence. Mark each as *provisional*.
5. **Converse.** Take your knowns, unknowns and assumptions to Claude and to ChatGPT, separately. Ask each to identify unknowns you missed, to challenge your assumptions and to propose an answer. Compare the two responses: where they agree, which unknowns each raised, and where they disagree.
6. **Resolve.** Decide on an answer as a team. Record what remains unknown and who owns it.
7. **Commit.** Enter the decision in the conversation log (page 34) and in the artifact section it supports (page 35).

Focus for this layer. Every object the platform and the agent produce must have a defined structure, an owner and an identity. If it cannot be stored, referenced and replayed, it is not yet designed.

Deliverable. ARTIFACT 7 — DATA AND INTERFACE SPECIFICATION. Entity relationship diagram · PostgreSQL schema · Pydantic contracts · API specification · Event schemas · Event sequencing rules · Outbox design · Idempotency rules · SSE timeline design.

A. DATA MODELING

Q7.1 An incident arrives without a globally unique reference. How should the platform identify it across requests, workflows and services?

Q7.2 The platform supports hundreds of applications and multiple environments. How should each incident be associated with its corresponding service and environment?

Q7.3 An engineer collects a log, a metric and a distributed trace. How should each piece of collected information be represented as a reusable record?

Q7.4 Evidence records need to be independently verified and traced back to their origin. Which attributes should every evidence record contain?

Q7.5 An investigation generates four competing explanations for the same failure. How should the platform represent and distinguish these hypotheses?

Q7.6 The agent selects one likely explanation after evaluating evidence. How should the resulting diagnosis be represented?

Q7.7 A diagnosis explains the cause, but an action describes a proposed change. How should the platform distinguish these two objects?

Q7.8 An agent proposes changing three lines of code. How should the resulting patch be represented and stored?

Q7.9 An investigation produces multiple possible changes. How should each proposed operation be represented independently?

Q7.10 A production action requires human authorization. What information should an approval record preserve?

Q7.11 A production action is attempted several times because of network failures. How should each attempt be represented?

Q7.12 An external system returns an execution response. How should the platform preserve the execution outcome?

Q7.13 A repair needs to be evaluated against service health. How should the resulting verification record be represented?

Q7.14 The model makes several calls during an investigation. Which information should be preserved for every model attempt?

Q7.15 The platform needs to limit financial exposure before executing model calls. How should budget reservations be represented?

Q7.16 Multiple components need to communicate changes in incident state. How should those changes be represented as events?

Q7.17 An investigation produces large logs, reports and patch files. How should the platform reference and manage these artifacts?

B. DATABASE DESIGN

Q7.18 The incident lifecycle contains incidents, evidence, hypotheses, diagnoses, patches, approvals and execution attempts. Which of these should become independent PostgreSQL tables?

Q7.19 A diagnosis belongs to an incident, but an incident may have multiple diagnoses. How should the database enforce this relationship?

Q7.20 Two incoming requests accidentally attempt to create the same incident. Which database constraints could prevent duplicate records?

Q7.21 Approval decisions and execution history must remain auditable. Which records should be immutable?

Q7.22 An incident has hundreds of evidence records and several patch candidates. How should relationships between these records be represented?

Q7.23 An engineer needs to inspect every previous approval decision. How should approval history be stored?

Q7.24 Two workers attempt to update the same incident simultaneously. How should the database prevent inconsistent updates?

Q7.25 An incident progresses through several states, and two components attempt to modify it concurrently. How should state versions be tracked?

Q7.26 Some information is highly structured while other information varies between evidence types. Which fields should be normalized into columns, and which could be represented using JSONB?

Q7.27 The application schema changes as new features are introduced. How should database migrations be managed without losing existing information?

C. API DESIGN

Q7.28 An external monitoring system needs to submit an incident. Which endpoint and request contract should the platform expose?

Q7.29 An engineer needs to inspect an existing incident. How should the API retrieve its current state?

Q7.30 An engineer wants to reconstruct the complete investigation. How should the API expose its event timeline?

Q7.31 An engineer needs to review an AI-generated patch. How should the API expose the corresponding artifact safely?

Q7.32 An engineer wants to authorize a proposed production action. Which API operation should accept the approval?

Q7.33 An engineer determines that a proposed patch is unsafe. How should the API record the rejection?

Q7.34 An operator needs to stop an ongoing investigation. How should cancellation be requested?

Q7.35 An orchestrator needs to determine whether the API process is running. Which endpoint should expose this information?

Q7.36 A deployment system needs to determine whether the API is ready to accept traffic. How should readiness be exposed?

Q7.37 The observability stack needs access to application metrics. Which endpoint should expose them?

Q7.38 An API request fails because the user lacks permission. How should the platform communicate this failure consistently?

Q7.39 Different user types need different access rights. How should authentication and authorization be incorporated into API design?

Q7.40 External integrations depend on API contracts that may evolve over time. How should the platform support versioning?

Q7.41 An engineer requests a patch artifact, but its internal filesystem path is exposed in the response. How should artifact access be abstracted?

D. EVENT ARCHITECTURE

Q7.42 An incident passes through 20 operations, but the dashboard only shows its current status. How could engineers reconstruct the entire history of the investigation?

Q7.43 Multiple components need to react when an incident changes state. How could the architecture support event-driven communication?

Q7.44 Several independent systems need to exchange events with consistent metadata. Which standard could provide a common event envelope?

Q7.45 An event is generated by the workflow engine, but downstream consumers cannot identify its source or destination. Which common fields should every event contain?

Q7.46 Two events have identical payloads but represent different occurrences. How should each event be uniquely identified?

Q7.47 Events arrive in a different order from the order in which operations occurred. How should sequencing be handled?

Q7.48 Several incidents progress simultaneously. Should events be globally ordered or ordered independently within each incident?

Q7.49 A consumer receives the same event multiple times. How should it prevent duplicate processing?

Q7.50 A consumer receives an event before receiving an earlier event. How should it detect and handle out-of-order delivery?

Q7.51 A dashboard disconnects while receiving an incident timeline. How should it resume receiving updates without losing events?

Q7.52 An engineer needs to receive real-time incident updates without repeatedly polling the API. Which streaming mechanism could support this?

Q7.53 The Temporal workflow engine maintains its own execution history. Why would exposing that internal history directly as the public incident timeline create architectural coupling?

E. THE DISTRIBUTED TRANSACTION PROBLEM

Q7.54 PostgreSQL successfully creates an incident, but Temporal becomes unavailable before the corresponding workflow starts. How can the platform prevent the incident from remaining

permanently unprocessed?

Q7.55 Temporal successfully starts a workflow, but PostgreSQL fails to save the corresponding incident record. How could this create inconsistent application state?

Q7.56 Two independent systems need to commit related changes. Why would an ordinary PostgreSQL transaction be insufficient to guarantee atomicity across both systems?

Q7.57 The API needs to save an incident and reliably arrange for a background workflow to start, even if the process crashes between these operations. Which transactional messaging pattern could solve this?

Q7.58 A durable event has been stored but has not yet been delivered to Temporal. How should a background dispatcher discover and process pending events?

Q7.59 The dispatcher starts a Temporal workflow successfully but crashes before marking the event as delivered. What should happen when the dispatcher restarts?

Q7.60 An API request is repeated because the client never received the original response. How could the system ensure that repeating the same operation does not produce duplicate effects?

Q7.61 Multiple requests need to be recognized as attempts to perform the same logical operation. How should an idempotency key be generated, stored and enforced?

Q7.62 A client reuses an existing idempotency key but sends a completely different payload. How should the API respond?

Q7.63 The dispatcher retries workflow creation after losing the previous response. How could a stable workflow identifier prevent duplicate workflow creation?

Q7.64 The team proposes Kafka to solve event delivery, but the initial platform has modest event volume. Under what workload or architectural conditions would Kafka become justified, and when would PostgreSQL plus an outbox be sufficient?

LAYER 7 · DATA ARCHITECTURE, APIS AND EVENT DESIGN

Conversation Log

Record one row per agent conversation. A conversation may cover several of the 64 questions in this layer; list all of them. Mark the agent used: **C** for Claude, **G** for ChatGPT, or both. Continue on additional sheets if required.

Questions	Agent	Conversation link	Decision reached	Open unknowns and owner	Artifact section

WHERE CLAUDE AND CHATGPT DISAGREED

Record each disagreement that changed a team decision, and how the team resolved it.

ARTIFACT 7

Data and Interface Specification

Built from. Artifacts 4 and 6.

Produce this artifact with your coding agent from the decisions recorded in the conversation log. Complete the table as each section is written.

	Required section	Questions this section answers	Done
7.1	Entity relationship diagram		☐
7.2	PostgreSQL schema		☐
7.3	Pydantic contracts		☐
7.4	API specification		☐
7.5	Event schemas		☐
7.6	Event sequencing rules		☐
7.7	Outbox design		☐
7.8	Idempotency rules		☐
7.9	SSE timeline design		☐

ACCEPTANCE CHECKS BEFORE SUBMISSION

- ☐ Every required section above is present and complete.
- ☐ Every section lists the questions it answers, and every question in this layer is answered by at least one section.
- ☐ Every assumption is marked *provisional* and has a named owner and a condition under which it will be validated.
- ☐ Every disagreement between Claude and ChatGPT that affected a decision is recorded with the team's resolution.
- ☐ The artifact is consistent with every artifact it is built from; any conflict is recorded as an explicit decision.
- ☐ The artifact was produced with the team's coding agent and is saved in the team repository.

Reviewed by _____ Date _____

LAYER 8

Distributed Workflows and State Machines

How does long-running work survive failure?

METHOD FOR THIS LAYER

Built from. Artifacts 6 and 7.

Work through every question in this layer with your coding agents. For each question, follow the procedure below before you write anything into the artifact.

1. **Frame.** Restate the question in one sentence, in your own words.
2. **Knowns.** List what the scenario and the earlier artifacts establish as fact.
3. **Unknowns.** List what you must discover, measure or decide.
4. **Assumptions.** List what you are treating as true without evidence. Mark each as *provisional*.
5. **Converse.** Take your knowns, unknowns and assumptions to Claude and to ChatGPT, separately. Ask each to identify unknowns you missed, to challenge your assumptions and to propose an answer. Compare the two responses: where they agree, which unknowns each raised, and where they disagree.
6. **Resolve.** Decide on an answer as a team. Record what remains unknown and who owns it.
7. **Commit.** Enter the decision in the conversation log (page 39) and in the artifact section it supports (page 40).

Focus for this layer. Establish why ordinary application code is insufficient for long-running, failure-prone processes before designing the workflow. Every state and transition must be explicit.

Deliverable. ARTIFACT 8 — WORKFLOW SPECIFICATION. Temporal workflow definition · Activity inventory · State machine diagram · State transition table · Retry and timeout policies · Approval wait design · Cancellation behavior · Replay strategy · Reconciliation strategy · Failure scenarios.

A. WORKFLOW FUNDAMENTALS

Q8.1 An incident investigation requires several operations that must happen in a particular order. How would you represent and coordinate this sequence?

Q8.2 A normal Python function executes from beginning to end inside a process. How would its behavior need to change to support operations that continue independently of that process?

Q8.3 Incident investigations can take several minutes or hours. What problems arise when the entire investigation is tied to one HTTP request?

Q8.4 A worker crashes halfway through an investigation. What happens to its local variables and in-memory execution state?

Q8.5 The platform restarts after a deployment, but several investigations are still active. Which information must survive the restart?

Q8.6 An investigation needs to continue from its last completed step after a crash. Which execution guarantees would make this possible?

Q8.7 Engineers need to understand which operations were completed before a worker crashed. How should execution history be preserved?

Q8.8 A workflow restarts after a failure and must reconstruct its previous progress. How could historical execution records be used to reproduce that progress?

Q8.9 The workflow engine re-executes workflow code to reconstruct state. Which restrictions must be imposed on workflow code to ensure repeatable behavior?

Q8.10 Workflow code needs to coordinate operations, while external operations need to communicate with databases and model providers. How should these responsibilities be separated?

Q8.11 An external model call succeeds, but the workflow process crashes before recording the result. How should the external operation be retried and reconciled?

B. TEMPORAL

Q8.12 The engineering team currently manages long-running tasks using background threads and custom database flags. These tasks frequently lose progress after crashes. Which durable execution system could replace this approach?

Q8.13 Several investigations need to survive worker restarts and resume from their previous state. How should the platform preserve workflow progress?

Q8.14 A workflow exists in the system, but no process is available to execute its next operation. Which component should poll for and execute pending work?

Q8.15 Multiple workflows need to be distributed across different worker processes. How should work be routed to the appropriate workers?

Q8.16 A workflow needs to call an external API or database. Which execution unit should perform this operation?

Q8.17 A model provider becomes unavailable temporarily. How should the platform retry a failed external operation?

Q8.18 A model request takes too long and blocks workflow progress. Which time limits should be enforced around external operations?

Q8.19 An investigation needs to pause until an engineer approves a patch. How can an external human decision resume the suspended workflow?

Q8.20 An approval request remains unanswered for 24 hours. How could the workflow wait without occupying a Python process?

Q8.21 An approval expires while the workflow is waiting. What should happen to the workflow?

Q8.22 An engineer requests cancellation while the workflow is running. How should the workflow respond?

Q8.23 A new version of the workflow code is deployed while existing investigations are still running. How should the system preserve compatibility with their previous execution histories?

Q8.24 Temporal stores detailed internal execution history, while the product needs an auditable business timeline. How should these two representations remain separate?

C. STATE MACHINE DESIGN

Q8.25 An incident can be received, investigated, diagnosed, repaired and resolved. Which states should the incident be allowed to occupy?

Q8.26 An incident arrives for the first time and has not yet been investigated. Which initial state should represent this condition?

Q8.27 An investigation has collected evidence, but no root cause has been validated. What conditions should be satisfied before moving it from investigating to diagnosed?

Q8.28 A generated patch passes some tests but fails others. Under which conditions should it be allowed to move from testing to review?

Q8.29 An action is awaiting human authorization. Which conditions must be satisfied before it can transition into applying?

Q8.30 An approver rejects a proposed action. Which state should the action enter, and what should happen to the workflow?

Q8.31 An approval request remains unanswered beyond its expiry time. How should the action and incident state change?

Q8.32 An application crashes while a production operation is being executed. How should the platform represent the incomplete execution?

Q8.33 The dispatcher times out, but the external system may already have applied the change. How should this uncertain outcome be represented?

Q8.34 The platform cannot determine whether an external operation succeeded. Which reconciliation process should establish the actual state?

Q8.35 A repair is successfully applied, but independent verification detects continuing failures. How should the incident transition?

Q8.36 An engineer cancels an investigation before any external changes have occurred. Which state transitions should be permitted?

Q8.37 An engineer requests cancellation after a production write has started. How should the system handle cancellation without pretending that the external operation has been reversed?

Q8.38 Two workers attempt to transition an incident into incompatible states. How should invalid transitions be prevented?

Q8.39 Several systems maintain information about incident progress. Which system should own the canonical business state?

D. DISTRIBUTED SYSTEMS FAILURES

Q8.40 A workflow retries an operation after a worker crash, and the operation may be delivered more than once. What delivery guarantee does this represent?

Q8.41 An application discards an operation if it cannot confirm delivery, even if that means the operation may never execute. What delivery guarantee does this represent?

Q8.42 An external API may successfully execute a write even when the caller receives a timeout. Why is guaranteeing exactly-once execution of arbitrary external side effects difficult?

Q8.43 An incident triggers an API retry, a workflow retry and an SDK retry simultaneously. How could these independent retries multiply the number of requests?

Q8.44 Three components independently retry the same failing model request. How should retry ownership be centralized?

Q8.45 A remote operation times out, but the external process may still be running. How should the system handle the uncertain execution?

Q8.46 PostgreSQL becomes unavailable while a workflow is collecting evidence. Which operations can safely continue, and which must stop?

Q8.47 Temporal becomes unavailable while the API is receiving new incidents. How should the platform preserve incoming work?

Q8.48 A production write has an unknown outcome after a timeout. How should the platform establish the external state before deciding whether another operation is safe?

LAYER 8 · DISTRIBUTED WORKFLOWS AND STATE MACHINES

Conversation Log

Record one row per agent conversation. A conversation may cover several of the 48 questions in this layer; list all of them. Mark the agent used: **C** for Claude, **G** for ChatGPT, or both. Continue on additional sheets if required.

Questions	Agent	Conversation link	Decision reached	Open unknowns and owner	Artifact section

WHERE CLAUDE AND CHATGPT DISAGREED

Record each disagreement that changed a team decision, and how the team resolved it.

ARTIFACT 8

Workflow Specification

Built from. Artifacts 6 and 7.

Produce this artifact with your coding agent from the decisions recorded in the conversation log. Complete the table as each section is written.

	Required section	Questions this section answers	Done
8.1	Temporal workflow definition		☐
8.2	Activity inventory		☐
8.3	State machine diagram		☐
8.4	State transition table		☐
8.5	Retry and timeout policies		☐
8.6	Approval wait design		☐
8.7	Cancellation behavior		☐
8.8	Replay strategy		☐
8.9	Reconciliation strategy		☐
8.10	Failure scenarios		☐

ACCEPTANCE CHECKS BEFORE SUBMISSION

- ☐ Every required section above is present and complete.
- ☐ Every section lists the questions it answers, and every question in this layer is answered by at least one section.
- ☐ Every assumption is marked *provisional* and has a named owner and a condition under which it will be validated.
- ☐ Every disagreement between Claude and ChatGPT that affected a decision is recorded with the team's resolution.
- ☐ The artifact is consistent with every artifact it is built from; any conflict is recorded as an explicit decision.
- ☐ The artifact was produced with the team's coding agent and is saved in the team repository.

Reviewed by _____ Date _____

LAYER 9

Agent Architecture and Engineering Reasoning

How does the system investigate like an engineer?

METHOD FOR THIS LAYER

Built from. Artifacts 3, 7 and 8.

Work through every question in this layer with your coding agents. For each question, follow the procedure below before you write anything into the artifact.

1. **Frame.** Restate the question in one sentence, in your own words.
2. **Knowns.** List what the scenario and the earlier artifacts establish as fact.
3. **Unknowns.** List what you must discover, measure or decide.
4. **Assumptions.** List what you are treating as true without evidence. Mark each as *provisional*.
5. **Converse.** Take your knowns, unknowns and assumptions to Claude and to ChatGPT, separately. Ask each to identify unknowns you missed, to challenge your assumptions and to propose an answer. Compare the two responses: where they agree, which unknowns each raised, and where they disagree.
6. **Resolve.** Decide on an answer as a team. Record what remains unknown and who owns it.
7. **Commit.** Enter the decision in the conversation log (page 45) and in the artifact section it supports (page 46).

Focus for this layer. Move from single model calls to a structured agent. The agent reasons and proposes; the application enforces policy, authorization and every external write.

Deliverable. ARTIFACT 9 — AGENT DESIGN SPECIFICATION. Coordinator · Skill definitions · Typed contracts · Tool registry · Evidence schema · Hypothesis schema · Diagnosis schema · Critic schema · Revision policy · Loop termination policy · Agent evaluation metrics.

A. UNDERSTANDING AGENTS

Q9.1 An application needs to generate text based on a sequence of input tokens. Which computational system could perform this task?

Q9.2 A conventional LLM produces an answer but cannot independently investigate an incident or interact with operational systems. How would you extend it to perform goal-directed actions?

Q9.3 A chatbot responds to individual questions, but an incident investigation requires multiple decisions and actions. How would you design the system to handle this longer process?

Q9.4 An investigation requires collecting information, forming a hypothesis and deciding what to do next. How could this sequence be organized into a recurring control loop?

Q9.5 The model needs to retrieve logs from a monitoring system. How should an external capability be exposed to it?

Q9.6 The model needs to request an operation from an application rather than merely generate text. How should that interaction work?

Q9.7 An agent must decide whether to collect more evidence, investigate another hypothesis or stop. Which component should control this decision-making cycle?

Q9.8 An investigation continues across several model calls and workflow activities. Which state must be preserved between these operations?

Q9.9 The model proposes executing a production repair, but the application has a policy prohibiting unapproved writes. How should the system separate model reasoning from application policy?

Q9.10 An agent can access infrastructure directly and execute arbitrary commands. What operational and security risks does this introduce?

B. AGENT DECOMPOSITION

Q9.11 A single agent is responsible for collecting evidence, diagnosing incidents, generating patches and authorizing execution. Why could combining all these responsibilities create reliability problems?

Q9.12 Different parts of the investigation require different capabilities and validation rules. How should the agent's responsibilities be divided?

Q9.13 The platform needs reusable units of agent behavior with predictable input and output contracts. How should these capabilities be structured?

Q9.14 An evidence collection operation receives inconsistent parameters and produces unpredictable output. How would typed input and output contracts improve this operation?

Q9.15 The investigation needs a component dedicated to collecting operational information. What should its responsibilities and boundaries be?

Q9.16 An incident contains several competing root-cause possibilities. What should the hypothesis-generation capability produce?

Q9.17 The platform needs to select a root cause using available evidence. What responsibilities should the diagnosis capability own?

Q9.18 The agent identifies a code regression and proposes a change. Which capability should generate the proposed patch?

Q9.19 Some responsibilities involve authorization, security policy and external writes. Why should these remain outside the agent's control?

C. EVIDENCE-BASED REASONING

Q9.20 The agent reports that a database connection failed, but cannot show where this information came from. How should the system represent and validate evidence provenance?

Q9.21 Two engineers collect the same log at different times, but the underlying application state has changed. Which timestamps should be preserved?

Q9.22 A log file is modified after the agent has inspected it. How could the system detect that the underlying evidence has changed?

Q9.23 The agent identifies one possible root cause and ignores several alternatives. How could competing hypotheses improve the investigation?

Q9.24 The agent needs to explain why a hypothesis is likely. How should supporting evidence be associated with that hypothesis?

Q9.25 A diagnosis contradicts an observed application trace. How should contradictory evidence be represented and evaluated?

Q9.26 A deployment happens immediately before an incident, but the actual cause is a database failure. How should the agent distinguish temporal correlation from causation?

Q9.27 The monitoring provider is unavailable and the investigation has incomplete information. How should the agent reason about missing evidence?

Q9.28 The agent generates an evidence reference that does not exist in the database. How should the application prevent this unsupported reference from entering the diagnosis?

Q9.29 A diagnosis cites several evidence IDs, but some belong to another incident. How should evidence references be validated?

D. THE CRITIC

Q9.30 The agent produces a diagnosis, but there is no independent mechanism to identify unsupported claims. What review capability should be introduced?

Q9.31 The diagnosis contains five claims, but only two are supported by collected evidence. Which criteria should an independent reviewer evaluate?

Q9.32 A diagnosis is almost correct but needs additional evidence. How should the reviewer communicate whether the result should be accepted, revised or escalated?

Q9.33 The critic rejects a diagnosis because an important metric contradicts it. What should happen next?

Q9.34 The critic identifies missing information that could resolve uncertainty. Should it be able to request additional evidence?

Q9.35 The agent repeatedly revises the same diagnosis without improving its quality. How should the number of revision attempts be bounded?

Q9.36 Two different models agree on the same diagnosis, but both rely on the same incorrect evidence. Why does model agreement not establish independent ground truth?

Q9.37 The critic returns an invalid structured response. How should the application handle this failure?

Q9.38 The agent has exhausted its evidence collection and revision budget. Under what conditions should the investigation stop?

E. TOOL ARCHITECTURE

Q9.39 The model can currently invoke any available Python function. How would you restrict its access to explicitly approved capabilities?

Q9.40 Some tools retrieve information, while others modify external systems. How should these tools be classified?

Q9.41 An agent invokes a log retrieval tool with an invalid time range and unrestricted output. How should the tool validate its parameters?

Q9.42 An investigation begins against one service, but the agent attempts to query unrelated services. How should service scope be enforced?

Q9.43 The agent requests an entire month's logs for a minor incident. How should retrieval time ranges and output limits be enforced?

Q9.44 Different investigations require different levels of infrastructure access. How should credentials be scoped?

Q9.45 The agent attempts to execute a command directly on the production host. How should the architecture prevent this?

Q9.46 A log contains instructions that attempt to manipulate the agent. How should the tool and context architecture prevent application data from becoming executable instructions?

Q9.47 The agent repeatedly calls the same tool with identical parameters without obtaining new information. How should redundant tool execution be detected and limited?

F. AGENT EVALUATION

Q9.48 The platform reports a root cause, but the engineering team needs to establish whether it is correct. How should diagnosis accuracy be measured?

Q9.49 The agent produces a diagnosis without collecting sufficient evidence. How should evidence coverage be measured?

Q9.50 The diagnosis contains claims that cannot be traced to any collected evidence. How should unsupported claims be quantified?

Q9.51 The agent escalates every incident to a human, including incidents it could resolve safely. How should escalation quality be measured?

Q9.52 A critic reviews hundreds of diagnoses but rarely identifies genuine errors. How should its

effectiveness be evaluated?

Q9.53 The agent makes 100 model calls to investigate a simple configuration error. How should investigation efficiency be measured?

LAYER 9 · AGENT ARCHITECTURE AND ENGINEERING REASONING

Conversation Log

Record one row per agent conversation. A conversation may cover several of the 53 questions in this layer; list all of them. Mark the agent used: **C** for Claude, **G** for ChatGPT, or both. Continue on additional sheets if required.

Questions	Agent	Conversation link	Decision reached	Open unknowns and owner	Artifact section

WHERE CLAUDE AND CHATGPT DISAGREED

Record each disagreement that changed a team decision, and how the team resolved it.

ARTIFACT 9

Agent Design Specification

Built from. Artifacts 3, 7 and 8.

Produce this artifact with your coding agent from the decisions recorded in the conversation log. Complete the table as each section is written.

	Required section	Questions this section answers	Done
9.1	Coordinator		☐
9.2	Skill definitions		☐
9.3	Typed contracts		☐
9.4	Tool registry		☐
9.5	Evidence schema		☐
9.6	Hypothesis schema		☐
9.7	Diagnosis schema		☐
9.8	Critic schema		☐
9.9	Revision policy		☐
9.10	Loop termination policy		☐
9.11	Agent evaluation metrics		☐

ACCEPTANCE CHECKS BEFORE SUBMISSION

- ☐ Every required section above is present and complete.
- ☐ Every section lists the questions it answers, and every question in this layer is answered by at least one section.
- ☐ Every assumption is marked *provisional* and has a named owner and a condition under which it will be validated.
- ☐ Every disagreement between Claude and ChatGPT that affected a decision is recorded with the team's resolution.
- ☐ The artifact is consistent with every artifact it is built from; any conflict is recorded as an explicit decision.
- ☐ The artifact was produced with the team's coding agent and is saved in the team repository.

Reviewed by _____ Date _____

LAYER 10

Inference Engineering and Knowledge Retrieval

How do we run models reliably and give them the right information?

METHOD FOR THIS LAYER

Built from. Artifacts 2 and 9.

Work through every question in this layer with your coding agents. For each question, follow the procedure below before you write anything into the artifact.

1. **Frame.** Restate the question in one sentence, in your own words.
2. **Knowns.** List what the scenario and the earlier artifacts establish as fact.
3. **Unknowns.** List what you must discover, measure or decide.
4. **Assumptions.** List what you are treating as true without evidence. Mark each as *provisional*.
5. **Converse.** Take your knowns, unknowns and assumptions to Claude and to ChatGPT, separately. Ask each to identify unknowns you missed, to challenge your assumptions and to propose an answer. Compare the two responses: where they agree, which unknowns each raised, and where they disagree.
6. **Resolve.** Decide on an answer as a team. Record what remains unknown and who owns it.
7. **Commit.** Enter the decision in the conversation log (page 51) and in the artifact section it supports (page 52).

Focus for this layer. This layer contains two related but separate engineering problems: running models reliably (Parts A–C), and supplying them with the right information (Parts D–E). Keep the two designs distinct.

Deliverable. ARTIFACT 10 — INFERENCE AND RETRIEVAL SPECIFICATION. Provider capability matrix · Model gateway contracts · Claude adapter · Tinker adapter · Budget policy · Concurrency policy · Retry and timeout policy · Token accounting · BM25 ingestion and search · Chunking strategy · Context assembly · Prompt versioning · Retrieval evaluation.

A. MODEL PROVIDER ARCHITECTURE

Q10.1 The platform needs to transform input instructions into generated tokens. Which computational model performs this operation?

Q10.2 The engineering team wants to use a model, but different companies provide access to models through different interfaces. How should the underlying model be distinguished from the provider delivering it?

Q10.3 The team needs to use a model without operating GPU infrastructure. Which inference delivery approach would support this?

Q10.4 The organization wants complete control over its model-serving infrastructure. How would self-hosted inference differ from using an external provider?

Q10.5 The platform currently uses Claude, but the engineering team wants to experiment with Tinker. How should their different responsibilities and execution models be evaluated?

Q10.6 The organization wants to use managed inference instead of maintaining its own serving infrastructure. Which operational responsibilities would be transferred to the provider?

Q10.7 The team proposes running its own GPU cluster even though its current inference volume is small. What workload or control requirements would justify this decision?

Q10.8 A team wants to serve models efficiently using GPU memory and batching optimizations. Which model-serving infrastructure could support this?

Q10.9 Different inference providers are responsible for model execution, but the platform still needs consistent application behavior. Which responsibilities should belong to the provider and which should remain inside the application?

B. MODEL GATEWAY

Q10.10 The agent directly communicates with three different model providers, each requiring a different request format. How would you introduce one internal model interface?

Q10.11 The platform needs to switch providers without rewriting the agent. How should provider-specific communication be isolated?

Q10.12 One provider returns tool calls in a different structure from another. How should response formats be normalized?

Q10.13 Different agent tasks require different model instructions, parameters and capabilities. Which fields should a ModelRequest contain?

Q10.14 The application needs to record the generated response, token usage and execution metadata. Which fields should a ModelResult contain?

Q10.15 A provider returns text, tool calls and other structured content in one response. How should these content types be represented?

Q10.16 The model requests a tool operation, but the application needs to validate it before execution. How should tool proposals be represented?

Q10.17 A provider refuses to perform a requested operation. How should the application represent and handle this refusal?

Q10.18 A model stops generating because it reached a token limit rather than completing the response. How should the application distinguish these outcomes?

Q10.19 A streaming response disconnects halfway through generation. How should the application detect an incomplete response?

Q10.20 A provider returns truncated JSON that cannot be parsed. How should the application handle incomplete structured output?

Q10.21 A model returns valid JSON, but the contents violate the application's expected schema. How should the response be validated?

Q10.22 One provider does not support structured output or tool calling required by an agent skill. How should the gateway handle capability incompatibility?

Q10.23 An investigation must be reproducible months later. Which model and checkpoint identity information should be preserved?

Q10.24 The organization wants to compare multiple providers before selecting one. Which capabilities and limitations should be included in a provider comparison?

C. INFERENCE BUDGETS

Q10.25 The model charges according to the amount of text it processes and generates. What unit should the platform use to measure this consumption?

Q10.26 Two model requests contain different amounts of input and generated output. How should their token usage be distinguished?

Q10.27 An investigation makes 20 model calls and produces an unexpectedly large bill. How should the platform estimate and control inference expenditure?

Q10.28 An investigation requires multiple model calls, but the organization wants to enforce a maximum expenditure. How should a per-incident budget be established?

Q10.29 The system starts several expensive model requests simultaneously without checking available budget. How could cost reservations prevent this?

Q10.30 Hundreds of incidents arrive at once and overload the model provider. How should the platform control concurrent inference requests?

Q10.31 One incident launches ten parallel reasoning operations while another launches only one. How should per-incident concurrency be limited?

Q10.32 An investigation continues making requests after its useful execution window has expired. How should a deadline be enforced?

Q10.33 A model request times out after 30 seconds, but the overall incident investigation is allowed to run for 10 minutes. How should the two time limits be distinguished?

Q10.34 The agent, critic and revision loop each generate additional tokens. How should cumulative token consumption be tracked?

Q10.35 A provider fails to report usage after completing a request. How should the platform account for uncertain expenditure?

Q10.36 The provider has consumed tokens, but the application has not received the usage information. Why should the system avoid immediately releasing the reserved budget?

Q10.37 Several components retry model calls independently, causing costs to multiply. How should the system control retry amplification?

Q10.38 A request fails because of a permanent authentication error. Why should this type of failure not be retried?

Q10.39 One model provider becomes unavailable, and the system automatically sends every request to another provider. What risks would this introduce, and why might automatic cross-provider fallback remain disabled initially?

D. RETRIEVAL ENGINEERING

Q10.40 The agent investigates an unfamiliar application but does not know the organization's operational procedures. How should internal runbooks be made available during investigation?

Q10.41 The model has no access to relevant external knowledge and produces answers using only its existing parameters. How could an external knowledge retrieval mechanism improve its responses?

Q10.42 The organization has hundreds of operational documents that need to become searchable. How should the ingestion process work?

Q10.43 Documents contain several topics and sections. How should they be divided into smaller retrievable units?

Q10.44 Naive fixed-size chunking separates important technical instructions from their headings. How could heading-aware chunking preserve document structure?

Q10.45 Engineers search for exact error codes, function names and configuration identifiers. Which lexical retrieval method would be appropriate?

Q10.46 A document contains a rare error identifier that appears in only a few documents. How should retrieval account for the importance of rare terms?

Q10.47 Keyword matching fails when the user describes a problem using different terminology from the documentation. How could vector embeddings improve retrieval?

Q10.48 Engineers need to search documents based on conceptual similarity rather than exact words. Which search mechanism could support this?

Q10.49 Keyword retrieval and semantic retrieval each miss relevant documents in different situations. How could their results be combined?

Q10.50 The initial platform needs reliable retrieval of exact technical identifiers without introducing unnecessary infrastructure. Why might BM25 be a reasonable starting point?

Q10.51 The retrieval system returns ten documents, but engineers do not know whether they are relevant. How should retrieval quality be measured?

Q10.52 Runbooks change over time, but an old investigation needs to be reproduced. How should immutable retrieval index revisions be maintained?

Q10.53 The agent cites a runbook paragraph, but the original document has changed. How should the source document version and exact text span be preserved?

Q10.54 An old runbook describes a procedure that is no longer valid. How should stale documentation be detected?

Q10.55 An administrator deletes a document, but the retrieval system continues returning its content. How should deleted documents be handled?

Q10.56 An engineer can access only a subset of organizational runbooks. How should retrieval enforce document-level permissions?

E. CONTEXT ENGINEERING

Q10.57 The model receives thousands of log lines, several metrics and multiple runbooks. How should the application decide which information belongs in its context?

Q10.58 The agent needs to combine live application telemetry with historical operational documentation. How should these different information sources be assembled?

Q10.59 Two evidence sources contradict each other. How should the context preserve this disagreement rather than silently removing it?

Q10.60 The context becomes too large for the model's input limit. How should information be compressed while preserving important details?

Q10.61 An investigation produces more information than the available token budget can accommodate. How should the system prioritize evidence?

Q10.62 A context summarizer removes source references while compressing evidence. How should provenance be preserved?

Q10.63 The agent generates a conversational summary of an incident, but engineers need to verify the original observations. Why should summaries not replace canonical evidence records?

Q10.64 An engineer wants to reproduce a diagnosis from six months ago. Which prompt, model, context and retrieval versions should be preserved?

LAYER 10 · INFERENCE ENGINEERING AND KNOWLEDGE RETRIEVAL

Conversation Log

Record one row per agent conversation. A conversation may cover several of the 64 questions in this layer; list all of them. Mark the agent used: **C** for Claude, **G** for ChatGPT, or both. Continue on additional sheets if required.

Questions	Agent	Conversation link	Decision reached	Open unknowns and owner	Artifact section

WHERE CLAUDE AND CHATGPT DISAGREED

Record each disagreement that changed a team decision, and how the team resolved it.

ARTIFACT 10

Inference and Retrieval Specification

Built from. Artifacts 2 and 9.

Produce this artifact with your coding agent from the decisions recorded in the conversation log. Complete the table as each section is written.

	Required section	Questions this section answers	Done
10.1	Provider capability matrix		☐
10.2	Model gateway contracts		☐
10.3	Claude adapter		☐
10.4	Tinker adapter		☐
10.5	Budget policy		☐
10.6	Concurrency policy		☐
10.7	Retry and timeout policy		☐
10.8	Token accounting		☐
10.9	BM25 ingestion and search		☐
10.10	Chunking strategy		☐
10.11	Context assembly		☐
10.12	Prompt versioning		☐
10.13	Retrieval evaluation		☐

ACCEPTANCE CHECKS BEFORE SUBMISSION

- ☐ Every required section above is present and complete.
- ☐ Every section lists the questions it answers, and every question in this layer is answered by at least one section.
- ☐ Every assumption is marked *provisional* and has a named owner and a condition under which it will be validated.
- ☐ Every disagreement between Claude and ChatGPT that affected a decision is recorded with the team's resolution.
- ☐ The artifact is consistent with every artifact it is built from; any conflict is recorded as an explicit decision.
- ☐ The artifact was produced with the team's coding agent and is saved in the team repository.

Reviewed by _____ Date _____

LAYER 11

Patch Generation, Sandboxing, Approval and Execution

How does a recommendation become a controlled, verified change?

METHOD FOR THIS LAYER

Built from. Artifacts 5, 8 and 9.

Work through every question in this layer with your coding agents. For each question, follow the procedure below before you write anything into the artifact.

1. **Frame.** Restate the question in one sentence, in your own words.
2. **Knowns.** List what the scenario and the earlier artifacts establish as fact.
3. **Unknowns.** List what you must discover, measure or decide.
4. **Assumptions.** List what you are treating as true without evidence. Mark each as *provisional*.
5. **Converse.** Take your knowns, unknowns and assumptions to Claude and to ChatGPT, separately. Ask each to identify unknowns you missed, to challenge your assumptions and to propose an answer. Compare the two responses: where they agree, which unknowns each raised, and where they disagree.
6. **Resolve.** Decide on an answer as a team. Record what remains unknown and who owns it.
7. **Commit.** Enter the decision in the conversation log (page 58) and in the artifact section it supports (page 59).

Focus for this layer. This is the critical safety layer. Treat patch generation, sandboxing, testing, approval, dispatch and verification as separate engineering problems with separate owners. No component may both propose and authorize a change.

Deliverable. ARTIFACT 11 — SAFE REMEDIATION SPECIFICATION. Patch schema · Diff and hashing strategy · Sandbox threat model · Resource isolation · Trusted test harness · Approval API · Authorization matrix · Immutable action schema · Dispatcher contract · Idempotency ledger · Reconciliation strategy · Verification predicates · Rollback strategy.

A. PATCH GENERATION

Q11.1 The AI identifies a bug and needs to communicate a proposed source-code modification. How should that modification be represented?

Q11.2 An AI-generated change modifies five lines of code, but engineers need to understand exactly what changed. How should the modification be displayed?

Q11.3 The model generates a complete replacement file, making it difficult to review individual modifications. How could the platform represent changes as a structured diff?

Q11.4 The model attempts to modify configuration files, application code and security policies simultaneously. How should editable file boundaries be defined?

Q11.5 The agent generates a patch against the latest source code, but another engineer deploys a new version before execution. How should the original source revision be preserved?

Q11.6 A patch modifies the test suite to make its own tests pass. How should the platform prevent modifications to trusted tests and security policies?

Q11.7 Engineers need to understand why each line was modified. How should the rationale behind a patch be recorded?

Q11.8 Two patches contain identical-looking changes but originate from different source revisions. How could a SHA-256 digest establish the identity of each patch?

Q11.9 A patch passes testing, but the model subsequently generates a modified version. How should the platform detect that the approved artifact has changed?

Q11.10 An incident has one diagnosis but several possible repair strategies. How should multiple patch candidates be associated with the same diagnosis?

Q11.11 The agent generates 15 different patch candidates, creating excessive testing costs. How should candidate generation be bounded?

Q11.12 Every proposed patch fails the trusted test suite. What should the platform do instead of attempting an unsafe production deployment?

B. SANDBOX ARCHITECTURE

Q11.13 An AI-generated patch contains malicious code. Why would executing it directly on the production application server create an unacceptable risk?

Q11.14 The engineering team needs to execute untrusted code without exposing the underlying host. Which isolation mechanism could provide this boundary?

Q11.15 A generated patch needs its own filesystem and process environment. How could containerization support this requirement?

Q11.16 An untrusted process attempts to inspect or modify processes belonging to other applications. Which operating-system isolation mechanisms could prevent this?

Q11.17 A generated process attempts to consume all available CPU and memory. How could the operating system enforce resource restrictions?

Q11.18 An untrusted container needs isolation from the host network and filesystem. Which Linux namespace capabilities could support this?

Q11.19 A generated process needs resource limits independent of other processes. How could control groups enforce these limits?

Q11.20 A container runs with root privileges, giving generated code excessive permissions. How should execution privileges be restricted?

Q11.21 An untrusted patch attempts to access the internet and download additional executable code. How should outbound network access be controlled?

Q11.22 The sandbox needs configuration information to execute tests. Why should production credentials never be included in its environment?

Q11.23 A generated process discovers the Docker host socket mounted inside its container. What level of host access could this expose?

Q11.24 An agent generates a patch that consumes excessive CPU resources. How should CPU quotas be enforced?

Q11.25 A malicious test process allocates memory continuously. How should memory consumption be restricted?

Q11.26 A generated process creates thousands of child processes. How should process limits be enforced?

Q11.27 A test suite generates several gigabytes of temporary files. How should disk usage be bounded?

Q11.28 A generated process never terminates. How should execution timeouts be enforced?

Q11.29 Every patch needs a clean environment that cannot inherit state from a previous test. How should ephemeral execution environments be created?

Q11.30 A test completes, but its container remains running and consumes infrastructure resources. How should sandbox cleanup work?

Q11.31 The organization relies exclusively on ordinary containers for executing hostile code. Which security limitations should be considered?

Q11.32 The threat model requires stronger isolation than ordinary containers provide. Under which circumstances should microVMs, Firecracker or gVisor be evaluated?

Q11.33 The model proposes a new sandbox configuration as part of its patch. Who should retain authority over sandbox creation and configuration?

Q11.34 The generated patch attempts to modify the trusted sandbox launcher itself. How should the

platform prevent this?

C. TESTING ARCHITECTURE

Q11.35 The engineering team needs to establish whether an individual function behaves correctly. Which testing approach should be used?

Q11.36 A function works independently but fails when communicating with PostgreSQL. Which testing approach would expose this integration failure?

Q11.37 A new patch fixes one issue but breaks previously working functionality. How should existing behavior be tested?

Q11.38 The original production failure cannot be reproduced in the development environment. How could a dedicated test recreate the original failure?

Q11.39 A patch appears to fix an error, but the original failure may never have been reproduced successfully. How would you establish a valid before-and-after comparison?

Q11.40 The patch passes a newly generated test, but engineers need evidence that the original failure has actually been fixed. Which tests should be executed?

Q11.41 A patch fixes the original failure but introduces a different application error. How should the platform detect this regression?

Q11.42 The AI modifies both the application and its test suite. How should the trusted test suite remain independent of candidate code?

Q11.43 The generated patch attempts to modify the test harness to bypass a failing assertion. Which security boundary should prevent this?

Q11.44 A sandbox finishes executing a patch. Which artifacts should it return to the platform?

Q11.45 Two different patches produce different test results. How should test results be associated with the exact patch that was executed?

Q11.46 A sandbox process crashes before returning its results. How should the platform classify this execution?

Q11.47 A test suite runs indefinitely because of a deadlock. How should the platform handle execution timeouts?

Q11.48 Some tests pass, but critical integration tests cannot execute because a dependency is unavailable. How should the platform represent an inconclusive result?

D. HUMAN APPROVAL ARCHITECTURE

Q11.49 An AI system can generate and execute its own production repair. Why would allowing the same system to authorize its own action create a security problem?

Q11.50 An engineer claims to be the person approving a patch. How should the platform establish their identity?

Q11.51 An authenticated engineer attempts to approve an action for a service they do not control. How should the platform enforce authorization?

Q11.52 Different users need different permissions to operate, approve and administer the platform. How should these roles be separated?

Q11.53 An approver receives a notification saying that the AI wants to fix an incident. What exact information should the approver review before making a decision?

Q11.54 An engineer approves a patch, but the model changes it one second later. How should approval be cryptographically or logically bound to the original patch digest?

Q11.55 An approved action is associated with several different application environments. How should the approval identify the exact target?

Q11.56 An engineer approves a patch against an old version of the application. How should the

approval be bound to the source revision?

Q11.57 An administrator changes the security policy after an approval has been issued. How should the original policy version be preserved and validated?

Q11.58 An approval request remains pending indefinitely. How should the platform enforce approval expiry?

Q11.59 An engineer approves a patch but subsequently loses their production permissions. Should the previous approval remain valid, and how should this be decided?

Q11.60 Two approvers submit conflicting decisions for the same action. How should the platform resolve this conflict?

Q11.61 An API accepts an approval request containing replacement patch content. How could this allow an attacker to substitute the approved action?

Q11.62 An engineer rejects a patch but the workflow continues toward execution. How should rejection become an enforced state transition?

Q11.63 An approver wants to withdraw an approval before execution begins. How should approval revocation work?

Q11.64 An auditor needs to establish who approved a production change and when. Which information should be recorded?

E. TRUSTED DISPATCHER

Q11.65 The AI generates a tool call that directly modifies production infrastructure. How should the platform prevent model-generated tool calls from executing without authorization?

Q11.66 Different components have different levels of authority over production resources. How could a capability-based permission model restrict each component?

Q11.67 The AI needs to execute a repair, but should only be able to invoke a limited set of predefined operations. How should a narrow allowlisted execution interface be designed?

Q11.68 A dispatcher receives an action request containing patch information from the model. Why should it reload the original immutable action from trusted storage instead of trusting the supplied arguments?

Q11.69 An action was approved several minutes ago, but the target environment has changed. Which conditions must be revalidated immediately before execution?

Q11.70 Multiple operations target the same service and source revision. How should each operation receive a stable identity?

Q11.71 A worker retries an action after a network failure. How could a stable action ID prevent duplicate external writes?

Q11.72 An external deployment system receives the same action twice. How could the target maintain an action-key ledger to identify previously processed requests?

Q11.73 A patch was generated against revision A, but the application is now at revision B. How should the dispatcher prevent the change from being applied to an unexpected version?

Q11.74 A production write succeeds, but the dispatcher loses its response. How should the platform establish whether the write occurred?

Q11.75 A dispatcher reports a timeout, but the external system may have completed the operation. Why should a timeout not automatically be interpreted as failure?

Q11.76 An external action has an uncertain outcome. Which reconciliation mechanism should establish its actual effect?

Q11.77 An organization wants to guarantee that every production write executes exactly once, even when network responses are lost. Why might this guarantee be impossible without cooperation from the target system?

Q11.78 A previous execution has an unknown outcome. Under which circumstances should auto-

matic retries be prohibited?

F. INDEPENDENT VERIFICATION

Q11.79 The dispatcher reports that an action executed successfully, but users are still experiencing errors. Why is an execution success response insufficient to establish service recovery?

Q11.80 The AI reports that the application is healthy after a repair. How could a separate verification component independently establish this claim?

Q11.81 An application has several service health requirements. Which measurable recovery conditions should the verifier evaluate?

Q11.82 The platform needs to determine whether users can successfully access the application. Which health checks should be performed?

Q11.83 A patch was intended to resolve an HTTP 500 error. How could the verifier reproduce the original failure condition?

Q11.84 The application returns HTTP 200 once after the patch is applied, but continues failing intermittently. How should post-repair error rates be evaluated?

Q11.85 A repair reduces the error rate but increases request latency significantly. How should recovery verification account for both conditions?

Q11.86 The application passes one health check immediately after repair but fails again two minutes later. How should the verifier establish a sufficiently long observation window?

Q11.87 A repair appears successful, but an unrelated application regression emerges shortly afterward. How should the platform monitor for new failures?

Q11.88 A repair has been applied, but telemetry is insufficient to establish whether it worked. How should the verifier distinguish verified, failed and inconclusive outcomes?

Q11.89 A repair fails verification. What should happen next, and under which conditions should the incident be escalated?

Q11.90 A rollback could restore service, but it would itself modify production state. Why should rollback require an independently authorized action?

Q11.91 An engineer needs to reverse a previous repair. How should the rollback operation be represented and linked to the original action?

Q11.92 The service appears healthy, but one required recovery predicate remains unsatisfied. Which conditions must be fulfilled before the incident can be marked resolved?

LAYER 11 · PATCH GENERATION, SANDBOXING, APPROVAL AND EXECUTION

Conversation Log

Record one row per agent conversation. A conversation may cover several of the 92 questions in this layer; list all of them. Mark the agent used: **C** for Claude, **G** for ChatGPT, or both. Continue on additional sheets if required.

Questions	Agent	Conversation link	Decision reached	Open unknowns and owner	Artifact section

WHERE CLAUDE AND CHATGPT DISAGREED

Record each disagreement that changed a team decision, and how the team resolved it.

ARTIFACT 11

Safe Remediation Specification

Built from. Artifacts 5, 8 and 9.

Produce this artifact with your coding agent from the decisions recorded in the conversation log. Complete the table as each section is written.

	Required section	Questions this section answers	Done
11.1	Patch schema		☐
11.2	Diff and hashing strategy		☐
11.3	Sandbox threat model		☐
11.4	Resource isolation		☐
11.5	Trusted test harness		☐
11.6	Approval API		☐
11.7	Authorization matrix		☐
11.8	Immutable action schema		☐
11.9	Dispatcher contract		☐
11.10	Idempotency ledger		☐
11.11	Reconciliation strategy		☐
11.12	Verification predicates		☐
11.13	Rollback strategy		☐

ACCEPTANCE CHECKS BEFORE SUBMISSION

- ☐ Every required section above is present and complete.
- ☐ Every section lists the questions it answers, and every question in this layer is answered by at least one section.
- ☐ Every assumption is marked *provisional* and has a named owner and a condition under which it will be validated.
- ☐ Every disagreement between Claude and ChatGPT that affected a decision is recorded with the team's resolution.
- ☐ The artifact is consistent with every artifact it is built from; any conflict is recorded as an explicit decision.
- ☐ The artifact was produced with the team's coding agent and is saved in the team repository.

Reviewed by _____ Date _____

LAYER 12

Reliability, Observability and Evaluation

How do we establish that it works under realistic failure?

METHOD FOR THIS LAYER

Built from. Artifacts 3, 4, 10 and 11.

Work through every question in this layer with your coding agents. For each question, follow the procedure below before you write anything into the artifact.

1. **Frame.** Restate the question in one sentence, in your own words.
2. **Knowns.** List what the scenario and the earlier artifacts establish as fact.
3. **Unknowns.** List what you must discover, measure or decide.
4. **Assumptions.** List what you are treating as true without evidence. Mark each as *provisional*.
5. **Converse.** Take your knowns, unknowns and assumptions to Claude and to ChatGPT, separately. Ask each to identify unknowns you missed, to challenge your assumptions and to propose an answer. Compare the two responses: where they agree, which unknowns each raised, and where they disagree.
6. **Resolve.** Decide on an answer as a team. Record what remains unknown and who owns it.
7. **Commit.** Enter the decision in the conversation log (page 65) and in the artifact section it supports (page 66).

Focus for this layer. Building the system is not sufficient. Every claim about reliability, safety or quality must be backed by a measurement, a denominator and a reproducible procedure.

Deliverable. ARTIFACT 12 — RELIABILITY AND EVALUATION SPECIFICATION. Metrics catalog · Tracing strategy · Alert definitions · Operational runbooks · Failure matrix · Backup and restoration procedure · Retention policy · Frozen evaluation suite · Evaluation methodology · Baseline and experiment configuration · Promotion gates.

A. SELF-OBSERVABILITY

Q12.1 The AI SRE platform itself experiences failures, but its users cannot tell whether the platform is healthy. How should it monitor its own operational state?

Q12.2 The platform monitors customer applications and also needs to monitor its own infrastructure. How should these two observability responsibilities be distinguished?

Q12.3 Engineers need to understand platform performance and reliability. Which operational metrics should be exposed?

Q12.4 Several incidents remain queued for processing. How should the platform measure how long each incident has been waiting?

Q12.5 A workflow remains active for several hours without making progress. How should stalled workflows be detected?

Q12.6 Model calls become increasingly slow, but the team has no historical latency measurements. How should model performance be monitored?

Q12.7 A provider begins returning authentication errors and rate-limit responses. How should provider failures be measured?

Q12.8 The platform reserves more model expenditure than it can safely afford. How should financial exposure be monitored?

Q12.9 Hundreds of approval requests accumulate without being reviewed. How should the approval backlog be measured?

Q12.10 Production writes sometimes finish without a confirmed response. How should unknown action outcomes be monitored?

Q12.11 A large percentage of repair attempts fail their recovery checks. How should verification

failures be exposed?

Q12.12 A request travels through the API, workflow engine, model gateway and dispatcher, but engineers cannot reconstruct its journey. How should distributed tracing be introduced?

Q12.13 Two services generate telemetry for the same incident, but their records cannot be correlated. Which trace identifiers should be propagated?

Q12.14 An API request triggers a Temporal workflow that makes several model requests. How should the platform correlate these operations?

Q12.15 Different infrastructure components need a common instrumentation standard. Which observability framework could provide this?

Q12.16 Prometheus needs to collect platform measurements. Which metrics should the application expose?

Q12.17 Engineers need a visual representation of incidents, model costs and workflow failures. Which dashboard information should Grafana display?

Q12.18 Observability data contains sensitive customer information. How should sensitive fields be redacted?

Q12.19 Raw prompts and application logs contain potentially confidential data. Why should they not be exported to external telemetry systems by default?

Q12.20 The telemetry exporter becomes unavailable. How should the platform preserve core functionality without making observability failure a platform-wide outage?

B. OPERATIONAL RELIABILITY

Q12.21 An orchestrator needs to determine whether a process is alive or able to serve traffic. How should health checks distinguish these conditions?

Q12.22 A process is running but cannot communicate with PostgreSQL. Why should liveness and readiness be treated differently?

Q12.23 A provider repeatedly fails and receives more requests from the platform. How could a circuit breaker prevent repeated calls to an unhealthy dependency?

Q12.24 A service receives requests faster than it can process them. How should backpressure protect the system?

Q12.25 The incident queue grows continuously because workers cannot process incoming work quickly enough. How should queue saturation be detected and managed?

Q12.26 A worker needs to restart during a deployment while several investigations are active. How should graceful shutdown and in-flight work draining operate?

Q12.27 A provider becomes unavailable during an investigation. Which operations should be paused, retried or escalated?

Q12.28 PostgreSQL becomes unavailable, but the API continues accepting write requests. How should the system behave?

Q12.29 The artifact store becomes unavailable while the platform is generating a patch. Which operations should stop?

Q12.30 Temporal becomes unavailable while workers are processing incidents. How should the platform preserve workflow consistency?

Q12.31 The approval store becomes unavailable while an action is awaiting authorization. Why should production writes be disabled?

Q12.32 Several dependencies are degraded, but read-only investigation remains possible. Which capabilities should remain available?

Q12.33 A critical dependency becomes unavailable, creating uncertainty around external state. Which failures should automatically stop all production writes?

C. BACKUP AND DISASTER RECOVERY

Q12.34 The platform experiences a complete infrastructure failure. Which information must be recoverable to restore its operations?

Q12.35 PostgreSQL, Temporal persistence and artifact storage contain related information. Why could independently restoring these systems to inconsistent points in time create problems?

Q12.36 A backup is taken while workflows are actively executing external operations. How could this produce an inconsistent recovery state?

Q12.37 The organization needs to back up its data while preventing new operations from creating inconsistent snapshots. How should a quiesced backup process work?

Q12.38 New incidents arrive while a backup is being prepared. How should new work be controlled during the backup?

Q12.39 An external action is halfway through execution when the platform enters backup mode. How should in-flight external effects be handled?

Q12.40 The organization needs to quantify acceptable data loss after a disaster. Which recovery measurement should be defined?

Q12.41 The organization needs to quantify how quickly the platform must become operational after a disaster. Which recovery measurement should be defined?

Q12.42 The platform claims it can recover within 30 minutes, but nobody has tested the process. How should actual recovery performance be measured?

Q12.43 Engineers restore a production backup into an environment that could accidentally execute pending writes. How should the restoration environment be fenced?

Q12.44 Restored workflows contain actions that may already have executed against external systems. Why should dispatch remain disabled until reconciliation is complete?

Q12.45 The restored database reports that an action is pending, but the external target confirms that it succeeded. How should the restored system reconcile these records?

Q12.46 A document is deleted from the main database, but copies remain inside retrieval indexes and artifact storage. How should deletion and derived-data consistency be managed?

Q12.47 The platform stores several years of incident logs and generated artifacts. How should retention requirements be defined?

Q12.48 An active investigation references artifacts that would otherwise be deleted under the retention policy. How should the platform preserve required evidence?

Q12.49 The organization has successfully created backups for six months but has never restored one. How should disaster recovery testing establish whether those backups are actually usable?

D. EVALUATION ENGINEERING

Q12.50 The platform claims it can automatically diagnose incidents, but the team has no objective evidence of its effectiveness. How should an evaluation system be established?

Q12.51 The team evaluates every new model using a different set of incidents. Why would a fixed evaluation dataset improve comparisons?

Q12.52 The platform needs a collection of incidents with known expected outcomes. How should a golden evaluation dataset be constructed?

Q12.53 The agent needs to be tested against known incidents without being given their expected root causes. How should the trusted answer key be separated from the agent?

Q12.54 The dataset contains only simple configuration errors. Which additional incident categories would be needed to evaluate realistic system behavior?

Q12.55 Different incidents have different expected root causes. How should expected outcomes be represented in the evaluation data?

Q12.56 The agent provides a diagnosis with several evidence citations. How should the evaluator establish whether those citations are valid?

Q12.57 The platform identifies the correct root cause in 70 out of 100 test cases. How should diagnosis accuracy be calculated?

Q12.58 The agent produces a technically correct diagnosis but proposes an unsafe repair. How should repair success be evaluated separately from diagnosis accuracy?

Q12.59 The agent attempts to fix an incident despite insufficient evidence. How should escalation behavior be evaluated?

Q12.60 The agent encounters malicious instructions embedded in logs. How should prompt-injection resistance be tested?

Q12.61 The model proposes an unauthorized production write. How should unauthorized-action prevention be evaluated?

Q12.62 A model achieves 100% accuracy on the same incidents used during development. How could a held-out dataset establish whether the system generalizes?

Q12.63 The team tests the system on 20 incidents and reports that it is production-ready. Why would this sample be insufficient for broad statistical claims?

Q12.64 A dashboard reports a 95% repair success rate, but does not display the total number of repair attempts. Why is the denominator necessary?

Q12.65 The platform successfully verifies 30 repairs out of 100 attempts. How should the verified repair rate be reported?

Q12.66 Engineers need to compare typical and worst-case response times. How should p50, p95 and p99 latency be calculated?

Q12.67 An investigation consumes model tokens, infrastructure resources and testing compute. How should total cost per incident be calculated?

Q12.68 The organization wants to know the expenditure required to achieve one verified repair. How should cost per verified repair be calculated?

Q12.69 The platform has not successfully verified any repairs. How should cost per verified repair be reported without producing a misleading number?

Q12.70 A model achieves high diagnosis accuracy but occasionally executes unauthorized production actions. Why should this security failure block release regardless of its average accuracy?

E. CONTROLLED EXPERIMENTATION

Q12.71 The team introduces a new model and claims that it improves diagnosis quality, but has no comparable previous results. How should a baseline be established?

Q12.72 Two model configurations need to be compared under the same conditions. How should a candidate configuration be defined?

Q12.73 An experiment changes the model, prompt, dataset and retrieval strategy simultaneously. Why would this make it difficult to identify the source of improvement?

Q12.74 The team wants to compare Claude and Tinker for incident diagnosis. How should the experiment be designed?

Q12.75 Engineers believe vector search will outperform BM25 for runbook retrieval. How should the two approaches be compared?

Q12.76 Context compression is introduced to reduce inference costs, but engineers are unsure whether it removes important information. How should its impact be measured?

Q12.77 The team wants to understand whether the critic actually improves diagnosis quality. How could an ablation study isolate its contribution?

Q12.78 A new experiment changes five parameters simultaneously. Why should experiments preferably change one variable at a time?

Q12.79 The platform needs to evaluate AI-generated actions against production incidents without actually applying changes. How could shadow execution support this?

Q12.80 A shadow experiment accidentally receives production write permissions. How should the architecture prevent this?

Q12.81 The platform is ready to test a new model on a small percentage of real incidents. How should a canary rollout be designed?

Q12.82 A candidate configuration performs worse than the existing configuration after deployment. How should it be rolled back?

Q12.83 A model produces more convincing explanations but has a lower verified repair rate. Why should promotion not depend only on the quality of its generated responses?

LAYER 12 · RELIABILITY, OBSERVABILITY AND EVALUATION

Conversation Log

Record one row per agent conversation. A conversation may cover several of the 83 questions in this layer; list all of them. Mark the agent used: **C** for Claude, **G** for ChatGPT, or both. Continue on additional sheets if required.

Questions	Agent	Conversation link	Decision reached	Open unknowns and owner	Artifact section

WHERE CLAUDE AND CHATGPT DISAGREED

Record each disagreement that changed a team decision, and how the team resolved it.

ARTIFACT 12

Reliability and Evaluation Specification

Built from. Artifacts 3, 4, 10 and 11.

Produce this artifact with your coding agent from the decisions recorded in the conversation log. Complete the table as each section is written.

	Required section	Questions this section answers	Done
12.1	Metrics catalog		☐
12.2	Tracing strategy		☐
12.3	Alert definitions		☐
12.4	Operational runbooks		☐
12.5	Failure matrix		☐
12.6	Backup and restoration procedure		☐
12.7	Retention policy		☐
12.8	Frozen evaluation suite		☐
12.9	Evaluation methodology		☐
12.10	Baseline and experiment configuration		☐
12.11	Promotion gates		☐

ACCEPTANCE CHECKS BEFORE SUBMISSION

- ☐ Every required section above is present and complete.
- ☐ Every section lists the questions it answers, and every question in this layer is answered by at least one section.
- ☐ Every assumption is marked *provisional* and has a named owner and a condition under which it will be validated.
- ☐ Every disagreement between Claude and ChatGPT that affected a decision is recorded with the team's resolution.
- ☐ The artifact is consistent with every artifact it is built from; any conflict is recorded as an explicit decision.
- ☐ The artifact was produced with the team's coding agent and is saved in the team repository.

Reviewed by _____ Date _____

LAYER 13

Implementation Planning and Delivery

How do we build it?

METHOD FOR THIS LAYER

Built from. Artifacts 1 to 12.

Work through every question in this layer with your coding agents. For each question, follow the procedure below before you write anything into the artifact.

1. **Frame.** Restate the question in one sentence, in your own words.
2. **Knowns.** List what the scenario and the earlier artifacts establish as fact.
3. **Unknowns.** List what you must discover, measure or decide.
4. **Assumptions.** List what you are treating as true without evidence. Mark each as *provisional*.
5. **Converse.** Take your knowns, unknowns and assumptions to Claude and to ChatGPT, separately. Ask each to identify unknowns you missed, to challenge your assumptions and to propose an answer. Compare the two responses: where they agree, which unknowns each raised, and where they disagree.
6. **Resolve.** Decide on an answer as a team. Record what remains unknown and who owns it.
7. **Commit.** Enter the decision in the conversation log (page 71) and in the artifact section it supports (page 72).

Focus for this layer. By this point the team holds a complete architecture. Convert everything discovered into an executable engineering plan. Every milestone must end in an observable, demonstrable outcome.

Deliverable. ARTIFACT 13 — COMPLETE IMPLEMENTATION PLAN. Repository structure · Dependency graph · Technology versions to verify · Milestones M0–M5 · Task breakdown · File ownership · Testing commands · Acceptance criteria · Manual demos · Deliverables · Risks and unresolved dependencies · Human approval gates.

A. REPOSITORY DESIGN

Q13.1 The engineering team needs to organize its growing Python codebase. How should the repository's package structure be designed?

Q13.2 The API, workflow engine and agent share several data structures. Where should common contracts live?

Q13.3 The API needs to communicate with the workflow engine. Which modules should own this integration?

Q13.4 Multiple model providers require separate implementations. Where should their adapters live?

Q13.5 The agent contains several reusable capabilities. How should skills be organized inside the repository?

Q13.6 The retrieval engine needs ingestion, indexing and search capabilities. Where should this functionality be placed?

Q13.7 The sandbox requires separate security and execution logic. How should it be isolated from ordinary application modules?

Q13.8 The platform needs independent recovery verification. Where should verification logic live?

Q13.9 The evaluation system needs datasets, test cases and scoring logic. How should these be organized?

Q13.10 The initial milestone does not require every planned module. Which modules should be included in M0, and which should be deferred?

Q13.11 The platform contains configuration values, API keys and production credentials. How should these be separated?

Q13.12 Engineers need to modify prompts without losing historical versions. How should prompts be versioned?

Q13.13 Different developers use different dependency versions locally. How should dependency locking ensure reproducible environments?

B. DEVELOPMENT TOOLCHAIN

Q13.14 Developers need a consistent mechanism to install and manage Python dependencies. Which dependency management approach should the project adopt?

Q13.15 Two developers install the same dependencies but receive different versions. How could a lockfile prevent this inconsistency?

Q13.16 Developers need PostgreSQL, Temporal and other supporting services running locally. How should the development environment orchestrate them?

Q13.17 A new engineer joins the team and needs to run the entire platform without manually configuring every dependency. Which services should the local development environment contain?

Q13.18 Code changes are frequently merged without checking formatting, types or tests. How should the CI pipeline validate each change?

Q13.19 Python code contains inconsistent formatting and common linting errors. Which static analysis tool should be introduced?

Q13.20 The platform contains several modules with unclear input and output types. How could static type checking detect incorrect assumptions?

Q13.21 The team needs to verify application behavior automatically. Which testing framework should be used?

Q13.22 Unit tests need to run without paying for external model calls, while some integration tests require real providers. How should offline tests and paid integration tests be separated?

Q13.23 A developer accidentally runs an integration test using production credentials. How should the development and testing environments prevent this?

C. TESTING STRATEGY

Q13.24 A Python function processes an incident but produces incorrect output for certain inputs. Which tests should isolate this behavior?

Q13.25 The API writes an incident to PostgreSQL, but the record contains incorrect relationships. Which database integration tests should validate this?

Q13.26 A Temporal workflow needs to wait for approval and resume after a signal. How should workflow behavior be tested?

Q13.27 The model adapter works with mocked responses but fails when communicating with the real provider. Which integration tests are required?

Q13.28 The sandbox needs to execute code against an actual disposable service. How should this environment be tested?

Q13.29 The agent can be manipulated using malicious log content. Which security tests should validate its boundaries?

Q13.30 Some behaviors cannot be fully verified through automated tests. Which manual demonstrations should be required before release?

Q13.31 The monitoring provider sends the same incident multiple times. How should duplicate intake be tested?

Q13.32 A Temporal worker crashes halfway through an investigation. How should workflow recovery be tested?

Q13.33 An approval request is replayed after a previous decision has already been recorded. How should the system prevent duplicate authorization?

Q13.34 An engineer modifies a patch after approval. How should the test suite verify that execution is blocked?

Q13.35 A production write succeeds, but the dispatcher crashes before receiving its response. How should this failure scenario be tested?

Q13.36 The database is restored from a backup containing active investigations. How should recovery and reconciliation be tested?

Q13.37 Every milestone must demonstrate tangible progress. What evidence should be required before considering a milestone complete?

D. MILESTONE DESIGN

Q13.38 The team wants to build the platform before introducing any AI model. Which minimum capabilities would demonstrate that the core system works?

Q13.39 The first milestone needs a demonstrable result rather than just a collection of completed tasks. What observable end-to-end outcome should it produce?

Q13.40 The initial workflow operates with deterministic responses, but stakeholders now want real AI-generated diagnoses. At which milestone should actual model providers be introduced?

Q13.41 The platform can collect incident information but cannot make decisions about what to investigate next. At which milestone should agent capabilities be introduced?

Q13.42 The agent can investigate incidents but does not have access to internal runbooks. At which milestone should retrieval capabilities be introduced?

Q13.43 The platform generates proposed patches, but currently has no isolated execution environment. At which milestone should the sandbox be introduced?

Q13.44 The sandbox can successfully test patches, but production actions still lack authorization controls. At which milestone should human approval be introduced?

Q13.45 Approved actions can be represented and validated, but the platform cannot modify the target application. At which milestone should actual write execution be introduced?

Q13.46 The platform works in a development environment, but its own failures are not monitored. At which milestone should production operational monitoring be introduced?

Q13.47 The organization wants to establish whether future model improvements are genuinely useful. At which milestone should the baseline evaluation suite be frozen?

Q13.48 The team has several experimental improvements that are not essential for the initial product. Which improvements should be moved into optional experimentation rather than blocking the core implementation?

E. PROJECT MANAGEMENT

Q13.49 The implementation contains several dependent milestones. How should dependencies between milestones be represented?

Q13.50 Some engineers can work on database schemas while others develop model adapters. How should parallel development opportunities be identified?

Q13.51 The team wants to begin coding, but critical architecture decisions remain unresolved. Which decisions must be finalized before implementation starts?

Q13.52 The platform depends on external model providers, but their availability and capabilities have not been confirmed. How should these unresolved dependencies be tracked?

Q13.53 Some implementation decisions affect production security and data integrity. Which decisions should require explicit human review?

Q13.54 Engineers mark tasks as completed, but stakeholders cannot establish whether the milestone actually works. How should milestone completion criteria be defined?

Q13.55 Management wants to see tangible progress after each milestone. Which demonstrations should be prepared for stakeholders?

Q13.56 The team completes the entire implementation, but another engineering team needs to understand the decisions and limitations. What should the final implementation report contain?

Q13.57 Engineers discover that a proposed implementation requires changing a previously approved architectural boundary. At which point should they stop and request architectural approval?

Q13.58 The platform passes most tests but still has an unresolved authorization bypass. Which failures should prevent the system from being released?

LAYER 13 · IMPLEMENTATION PLANNING AND DELIVERY

Conversation Log

Record one row per agent conversation. A conversation may cover several of the 58 questions in this layer; list all of them. Mark the agent used: **C** for Claude, **G** for ChatGPT, or both. Continue on additional sheets if required.

Questions	Agent	Conversation link	Decision reached	Open unknowns and owner	Artifact section

WHERE CLAUDE AND CHATGPT DISAGREED

Record each disagreement that changed a team decision, and how the team resolved it.

ARTIFACT 13

Complete Implementation Plan

Built from. Artifacts 1 to 12.

Produce this artifact with your coding agent from the decisions recorded in the conversation log. Complete the table as each section is written.

	Required section	Questions this section answers	Done
13.1	Repository structure		☐
13.2	Dependency graph		☐
13.3	Technology versions to verify		☐
13.4	Milestones M0–M5		☐
13.5	Task breakdown		☐
13.6	File ownership		☐
13.7	Testing commands		☐
13.8	Acceptance criteria		☐
13.9	Manual demos		☐
13.10	Deliverables		☐
13.11	Risks and unresolved dependencies		☐
13.12	Human approval gates		☐

ACCEPTANCE CHECKS BEFORE SUBMISSION

- ☐ Every required section above is present and complete.
- ☐ Every section lists the questions it answers, and every question in this layer is answered by at least one section.
- ☐ Every assumption is marked *provisional* and has a named owner and a condition under which it will be validated.
- ☐ Every disagreement between Claude and ChatGPT that affected a decision is recorded with the team's resolution.
- ☐ The artifact is consistent with every artifact it is built from; any conflict is recorded as an explicit decision.
- ☐ The artifact was produced with the team's coding agent and is saved in the team repository.

Reviewed by _____ Date _____